

# Hybrid Sketching Methods for Dynamic Connectivity on Sparse Graphs

Quinten De Man\*  
University of Maryland  
College Park, MD, USA  
deman@umd.edu

Gilvir Gill\*  
Stony Brook University  
Stony Brook, NY, USA  
gigill@cs.stonybrook.edu

Michael A. Bender  
Stony Brook University  
Stony Brook, NY, USA  
bender@cs.stonybrook.edu

Laxman Dhulipala  
University of Maryland  
College Park, MD, USA  
laxman@umd.edu

David Tench  
Lawrence Berkeley National Lab  
Berkeley, CA, USA  
dtench@pm.me

## Abstract

Dynamic connectivity is arguably the most basic and fundamental dynamic graph problem, and recent algorithmic breakthroughs on *dynamic graph sketching* have reshaped what is theoretically possible for the problem: by encoding the graph as per-vertex linear sketches, these algorithms solve dynamic connectivity in only  $\Theta(V \log^2 V)$  space, independent of the number of edges, with an asymptotic advantage over lossless  $\Theta(V + E)$ -space structures that grows as the graph becomes denser. Prior to this work, no practical dynamic connectivity algorithm has been able to translate these theoretical breakthroughs into space savings on real-world graphs. The main obstacle is that per-vertex sketches cost thousands of bytes per vertex, so sketching only pays off once the graph becomes extremely dense.

Our starting observation is that real-world graphs that are sparse on average are often not uniformly sparse—these graphs can contain dense cores on a small subset of the vertices that accounts for a large fraction of the edges. We exploit this structure via *hybrid sketching*: sketch only the dense core, and store the sparse periphery losslessly.

On the theoretical side, we design new hybrid algorithms for both fully-dynamic and semi-streaming connectivity with space  $O(\min\{V + E, V \log V \log(2 + E/V)\})$  w.h.p., simultaneously matching the lossless bound on sparse graphs, the sketching bound on dense graphs, and improving on both in an intermediate regime. A key technical ingredient is BALLOONSKETCH, a new  $\ell_0$ -sampler that reduces per-vertex sketch sizes by up to  $8\times$  on real-world graphs. We give a practical C++ implementation of our new algorithms in HYBRIDSCALE, a modular system that treats the lossless and sketch-based components as subroutines. To our knowledge, HYBRIDSCALE is the first sketch-based dynamic connectivity system to save space on commonly studied real-world graphs. Compared to the state-of-the-art lossless baseline, HYBRIDSCALE uses up to 15% less space on sparse real-world graphs (average degree  $< 100$ ), up to 92% less space on intermediate density graphs (average degree  $\approx 100$ – $1000$ ), and up to 97% less space on synthetic dense graphs (average degree  $> 1000$ ).

## 1 Introduction

Graph data and graph algorithms play a fundamental role in modern computing, underlying applications in web search, transactional systems, and scientific computing, among many other areas. In nearly all of these settings, the underlying graph is inherently dynamic:

edges are continually inserted and deleted into the graph as the state of the system evolves (e.g., as transactions are performed). Maintaining basic structural queries on a graph as it changes is therefore a foundational primitive for downstream analytics, and *dynamic connectivity* (answering whether two vertices currently lie in the same connected component as the edge set changes) is arguably the most fundamental and well-studied dynamic primitive [16, 26, 29, 30, 32–37, 46, 57, 59]. Despite a four-decade old theoretical foundation, translating these results to practical systems has proven extremely difficult, and dynamic connectivity remains an active frontier for both theoretical and practical research.

Recent experimental work on dynamic connectivity pursues two broad approaches, each with distinct trade-offs. The first approach is using *lossless* dynamic data structures where every edge in the graph is maintained within the structure. Lossless systems use  $\Theta(V + E)$  space and are reasonably space-efficient for sparse inputs where  $E$  is not much larger than  $V$ . The second approach is to use *graph sketching*, which encodes the graph as a collection of vertex-level linear sketches and solves dynamic connectivity in  $\Theta(V \log^2 V)$  space, i.e., in space that is not dependent on the number of edges. The attraction and promise of graph sketching is that it remains space-efficient even as the graph becomes arbitrarily dense, while lossless methods would continue to use space linear in the number of edges.

Sketching techniques have found massive success in applications across computer science, particularly in streaming algorithms for networking and data analytics where sketches can exponentially reduce the space requirements of fundamental analytical tasks [5, 19, 25, 38]. However, in *graph sketching* each vertex requires its own sketch, so the *total* space requirement is significantly higher than the polylogarithmic space achieved by sketching in other domains.

Specifically, per-vertex graph sketches require  $\Theta(\text{polylog}(V))$  words, translating to thousands of bytes per vertex. Thus, sketching only beats storing the edges in an adjacency list when the average degree is at least in the thousands. However, the average degrees of real-world graphs are typically at most in the hundreds, which is at least an order of magnitude below this break-even point when every vertex is sketched. For example, on the Orkut social media graph ( $V \approx 3.07$  million, average-degree 76.3) [43], the state-of-the-art sketch-based dynamic connectivity system, CUPCaKE [22], would require roughly  $36\times$  more space than the state-of-the-art lossless system, an implementation of Cluster Forest [21].

\*Both authors contributed equally to this research.

Despite an elegant and influential body of theory [15, 44, 58], existing sketching systems [55, 56] have demonstrated space savings only on *synthetic* dense inputs such as Kronecker graphs and high-density Erdős–Rényi graphs. In this paper we study whether we can carefully leverage graph sketching to obtain more space-efficient dynamic connectivity data structures even on sparse real-world datasets.

Crucially, graphs that are sparse on average are typically not *uniformly* sparse. For example, social networks, web graphs, citation graphs, and collaboration networks, among others, are sparse on average but often contain a small, densely interconnected *core* that accounts for a large fraction of the edges [6, 8, 11, 14, 27, 39, 43, 47, 50]. On such graphs, the cost of sketching vertices in the core would be worthwhile, but would be prohibitive if blindly applied to every vertex. This observation suggests a natural strategy: sketch only the dense core, and store the sparse periphery losslessly. We refer to this approach as *hybrid sketching*. Although the idea is straightforward to state, realizing it both in theory and in practice requires carefully understanding how to combine each of the three moving parts (the graph sketching system, the lossless structure, and the logic that routes edges between them). There are three major challenges in the way of designing a hybrid sketching method:

- (1) We must find a space-minimizing partition of the graph into a sketched core and lossless periphery with no prior structural knowledge. We also desire high update throughput and low query latency. Further, the partitioning scheme must keep the lossless and sketch representations consistent.
- (2) Since the graph is dynamic, the boundary between the dense core and the sparse periphery may change over time. Moving edges from the core to the periphery requires recovering them from the sketch data structure. However, classical connectivity sketches do not guarantee returning all edges, only some of the sketched support. Hybrid sketching therefore requires leveraging a sparse recovery sketch that returns the full support of a sparse vector with high probability.
- (3) The state-of-the-art implementations of  $\ell_0$ -samplers, the main building block underlying graph sketching, have a space overhead of thousands of bytes per-vertex. As a result, these sketches don't save space over a lossless representation until the degree is in the thousands. Commonly-studied real-world graphs rarely have average degree higher than the low hundreds, so asymptotically smaller  $\ell_0$ -samplers are likely required to save space.

In this paper we introduce theoretically-efficient and practical techniques for implementing hybrid sketching algorithms, together with an accompanying system that sketches only the dense portion of a graph while storing the rest losslessly. To address the first challenge, we develop a hybrid framework that routes edges between a sketched dense core and a losslessly stored sparse periphery as the graph evolves, preserving the semantics of connectivity queries end-to-end. Regarding the second challenge, works on Invertible Bloom Lookup Tables (IBLTs) provide a sketching primitive to recover the entire support of a sparse vector with high probability [9, 31]. To address the third challenge, we design BALLOONSKETCH, an  $\ell_0$ -sampler whose aggregate space usage tracks the true support size of the sketched vector rather than the universe size, improving on the best known  $\ell_0$ -sampling space bound when the vector is sparse.

Taken together, these contributions yield the first hybrid sketching algorithms for dynamic connectivity whose space usage simultaneously matches the lossless bound on sparse inputs, matches the sketching bound on dense inputs, and improves on both in the intermediate regime. We realize these algorithms in a system, HYBRID-SCALE<sup>1</sup>, which to our knowledge is the first sketch-based dynamic connectivity system to deliver space savings on real-world graphs. Next, we give a more technical description of our contributions.

## 1.1 Our Contributions

Our main theoretical results are new hybrid sketching algorithms for graph connectivity with strong space complexity on both sparse and dense graphs. Their key property is that their space complexity simultaneously matches the asymptotic space of lossless methods on sparse inputs and of sketching-based methods on dense inputs, and improves on both in the intermediate regime.

**THEOREM 1.1 (HYBRID DYNAMIC CONNECTIVITY).** *There exists a dynamic connectivity algorithm with space complexity*

$$O\left(\min\left\{V + E, V \log(V) \log\left(2 + \frac{E}{V}\right)\right\}\right) \text{ w.h.p.}$$

*The update complexity is amortized  $O(\log^4 V)$ . The query complexity is worst-case  $O(\log V / \log \log V)$ .*

We additionally give a hybrid algorithm for the *semi-streaming* model, in which the graph arrives as a stream of edge insertions and deletions under an  $O(V \text{polylog}(V))$  memory budget, and connectivity queries are answered from the final sketch rather than continuously during the stream [3, 4, 28, 42, 45]. This is the classical setting in which graph sketching was originally developed, and it remains the standard benchmark for evaluating sketch-based space complexity. Our algorithm achieves the same hybrid space bound as in the fully-dynamic setting, with asymptotically faster update complexity in exchange for slower one-shot queries.

**THEOREM 1.2 (HYBRID STREAMING CONNECTIVITY).** *There exists a semi-streaming algorithm for connectivity on polynomial-length input streams of edge updates with space complexity*

$$O\left(\min\left\{V + E, V \log(V) \log\left(2 + \frac{E}{V}\right)\right\}\right) \text{ w.h.p.}$$

*The update complexity is amortized  $\Theta(\log V)$  w.h.p. The query complexity is worst-case  $\Theta(V \log^2 V)$ .*

The hybrid bounds above rest on a new sketching primitive, BALLOONSKETCH, which we believe is of independent interest. This result improves on the best known  $\ell_0$ -samplers, which use  $\Theta(L \log n)$  space, by replacing the universe-size dependence with a dependence on the true support size  $m$ . This improved sketch algorithm results in up to a 8× reduction in per-vertex sketch size on sparse real-world graphs. We find that this space reduction is necessary for our hybrid sketching algorithm to be space-efficient on these inputs.

**THEOREM 1.3 (BALLOONSKETCH).** *There exists an  $\ell_0$ -sampler, BALLOONSKETCH, such that given  $L$  independent BALLOONSKETCHES, each sketching the same vector  $x \in \mathbb{F}^n$  with  $m = \|x\|_0$ :*

- (1) *The total space usage of the sketches is  $O(L \log m + \log n)$  w.h.p.*
- (2) *The total cost of one update to each sketch is  $O(L + \log n)$  w.h.p.*

<sup>1</sup>Code is at <https://github.com/GraphStreamingProject/HybridDynamicQueriesCC>.

**The HYBRIDSCALE system.** We realize the hybrid framework in HYBRIDSCALE, a modular dynamic connectivity system whose design separates the lossless and sketch-based components. Any lossless dynamic connectivity structure can be plugged in on one side, and any sketch-based one on the other. Our implementation uses an implementation of Cluster Forest [1, 2, 21] on the lossless side. For the sketch-based side, we implemented a new system, BALLOONDC, combining ideas from Gibb’s algorithm [30], CUPCaKE [22], and our new sketching primitive BALLOONSKETCH.

To our knowledge, HYBRIDSCALE is the first graph sketching system to deliver space savings on commonly studied real-world graphs. Compared to the state-of-the-art lossless baseline, HYBRIDSCALE uses up to 15% less space on sparse real-world graphs (average degree  $< 100$ ), up to 92% less space on intermediate density graphs (average degree  $\approx 100$ –1000), and up to 97% less space on synthetic dense graphs (average degree  $> 1000$ ). Update and query throughput remain competitive with both pure baselines.

## 2 Preliminaries

### 2.1 Semi-Streaming and Dynamic Graph Models

In the *graph semi-streaming model* [28], an algorithm is presented with a *stream* of edge insertion or deletion updates that define a graph. The challenge in this model is to compute some property of the graph given a single pass over the stream and memory sublinear in the size of the graph. Specifically, stream  $S$  defines a graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  with  $V = |\mathcal{V}|$  and  $E = |\mathcal{E}|$ . Each update has the form  $((u, v), \Delta)$  where  $u, v \in \mathcal{V}$ ,  $u \neq v$  and  $\Delta \in \{-1, 1\}$  where 1 indicates an edge insertion and  $-1$  indicates an edge deletion. In this paper we study the *semi-streaming connected components problem* which returns a spanning forest of the graph at the end of the stream.

In the *dynamic graph model*, the challenge is to be able to efficiently query a specific structural property of a changing graph at any time. Specifically the algorithm is given a sequence  $S$  of updates (INSERTEDGE( $e$ ) or DELETEEDGE( $e$ )) which must be processed in order. Any prefix of the first  $i$  updates in  $S$  defines a graph  $\mathcal{G}_i = (\mathcal{V}_i, \mathcal{E}_i)$ . Immediately after processing the  $i$ -th update, one or more queries may be issued and the algorithm must compute the answers for graph  $\mathcal{G}_i$  before receiving the  $i + 1$ -th update. In this paper we study the *dynamic connectivity problem*, where the structural property of interest is connectivity, and queries of the form CONNECTED( $u, v$ ) return whether vertices  $u$  and  $v$  are connected in  $\mathcal{G}_i$ .

### 2.2 Linear Sketching for Streaming Connectivity

Ahn. et. al. ([3]) introduced the first semi-streaming algorithm for connected components using  $O(V \log^2 V)$  space. The core innovation of this approach is that the algorithm represents each vertex’s adjacency list as a compact *linear sketch* of its characteristic vector.

For a vertex  $v$ , its *characteristic vector*  $f_v \in \mathbb{R}^{\binom{V}{2}}$  encodes its incident edges such that for any edge  $(u, w)$ , the entry  $f_v((u, w))$  is 1 if  $v = u$  and  $(u, w) \in \mathcal{E}$ ,  $-1$  if  $v = w$  and  $(u, w) \in \mathcal{E}$ , and 0 otherwise. This specific encoding ensures that for any subset of vertices  $S \subseteq \mathcal{V}$ , the sum of their vectors  $\sum_{v \in S} f_v$  isolates the cut  $(S, \mathcal{V} \setminus S)$ . Every internal edge within  $S$  is added twice and cancels out perfectly, leaving only the edges that cross between  $S$  and the rest of the graph. Storing characteristic vectors explicitly would require  $O(V^3)$  total space. To solve this, the streaming algorithm compresses them using an  *$\ell_0$ -sampler*.

**DEFINITION 2.1.** An algorithm  $\mathcal{S}$  is an  *$\ell_0$ -sampler* with failure probability  $\delta \in (0, 1)$  if it can process coordinate updates to a vector  $x$  and maintain a compact sketch  $\mathcal{S}(x)$  satisfying:

- (1) **Sampleable:** If  $x$  is non-zero, sampling the sketch returns a uniform random non-zero coordinate with probability at least  $1 - \delta$ .
- (2) **Linear:** For any vectors  $x$  and  $y$ ,  $\mathcal{S}(x) + \lambda \mathcal{S}(y) = \mathcal{S}(x + \lambda y)$ .

With constant  $\delta$ , the  $\ell_0$ -sampler from Ahn et al. [3] has space, update, and sampling complexity  $O(\log V)$ . In Section 3.1 we provide more details about a more recent  $\ell_0$ -sampler, CAMEOSKETCH [56], as well as our new  $\ell_0$ -sampler, BALLOONSKETCH.

Because  $\ell_0$ -samplers are linear, the streaming algorithm can compute an  $\ell_0$ -sampler for an entire component’s cut simply by summing the sketches of its individual vertices:  $\sum_{v \in S} \mathcal{S}(f_v)$ . By maintaining  $\Theta(\log V)$  independent sketches per vertex, the algorithm can sum the sketches of any merged component (supernode) to successfully sample an outgoing edge. Repeating this process simulates the  $\Theta(\log V)$  rounds of Borůvka’s algorithm required to recover a spanning forest.

Tench et. al. provided the first practical implementation of the AGM sketch in GRAPHZEPPELIN [55] by introducing CUBESKETCH, a specialized  $\ell_0$ -sampler that operates over  $\mathbb{F}_2$  rather than  $\mathbb{Z}$ . The current state-of-the-art  $\ell_0$ -sampler, CAMEOSKETCH [56] (see Section 3.1), builds on this foundation by further improving CUBESKETCH’s worst-case running time and sampling success probability.

### 2.3 Dynamic Connectivity

**Lossless Dynamic Connectivity.** Traditional dynamic connectivity algorithms all store a data structure containing a lossless representation of the graph. We refer to these as lossless dynamic connectivity algorithms. The best algorithms, such as Cluster Forest [59], have  $O(V + E)$  space usage,  $O(\log^2 V)$  or better (amortized) update time, and  $O(\log V)$  or better query time. De Man et al. [21] provided a practical implementation of the Cluster Forest algorithm.

**Sketch-Based Dynamic Connectivity.** More recently, a new class of dynamic connectivity algorithms has emerged which combines both the polylogarithmic update and query times of traditional lossless dynamic connectivity algorithms and the small space usage of streaming connectivity algorithms [30, 37]. We call these sketch-based dynamic connectivity algorithms. The best known sketch-based algorithm, Gibb’s algorithm [30], uses  $O(V \log^2 V)$  space, processes updates in  $O(\log^4 V)$  time, and answers queries in  $O(\log V / \log \log V)$  time. Each query is correct w.h.p.

The core component of Gibb’s algorithm is the *cutset* [37]. For a graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  and a spanning forest  $\mathcal{F} = (\mathcal{V}, \mathcal{E}_{\mathcal{F}})$ , it can find an edge crossing a component’s cut during dynamic updates. Letting  $C(w)$  be the forest component containing  $w$ , a cutset supports:

- **LINK( $u, v$ ):** Insert edge  $(u, v) \notin \mathcal{E}_{\mathcal{F}}$  into  $\mathcal{F}$ , merging  $C(u)/C(v)$ .
- **CUT( $u, v$ ):** Remove edge  $(u, v) \in \mathcal{E}_{\mathcal{F}}$  from  $\mathcal{F}$ , splitting  $C(u, v)$ .
- **UPDATE( $e$ ):** Insert edge  $e$  into  $\mathcal{E}$  (if absent) or delete it (if present).
- **QUERY( $v$ ):** Return an edge crossing the cut of  $C(v)$  with constant success probability  $p$ .

Cutsets are implemented via a dynamic tree augmented with an  $\ell_0$ -sampler  $\mathcal{S}(f_v)$  for each vertex  $v$ . The dynamic tree must support subtree aggregate queries using a commutative, associative function (in this case sketch addition). Due to  $\ell_0$ -sampler linearity, querying a component’s subtree returns the sum of its vertices’ sketches, which

encodes the cut between the component and the rest of the graph. Cutsets typically use  $O(V \log V)$  space, perform LINKS, CUTS, and UPDATES in  $O(\log^2 V)$  time, and QUERY in  $O(\log V)$  time.

Gibb’s algorithm maintains cutsets  $F_0, \dots, F_{top}$  (where  $top = \Theta(\log V)$ ) across successive forest levels  $\mathcal{F}_0 \subseteq \dots \subseteq \mathcal{F}_{top}$ . Higher levels track increasingly larger components, ultimately yielding the true connected components. It also maintains a separate dynamic tree  $\mathcal{T}$  supporting maximum-weight path queries, where tree edges match  $\mathcal{E}_{\mathcal{F}_{top}}$  and an edge’s weight is the lowest level  $i$  that the edge appears in  $\mathcal{F}_i$ . The algorithm ensures three invariants:

- (1)  $\mathcal{F}_0 = (\mathcal{V}, \emptyset)$ .
- (2)  $\mathcal{F}_i \subseteq \mathcal{F}_{i+1}$  for  $0 \leq i < top$ .
- (3) For  $0 \leq i < top$ , if a QUERY on  $F_i$  for a component  $C \in \mathcal{F}_i$  succeeds,  $C$  is a proper subset of some  $C' \in \mathcal{F}_{i+1}$ .

Gibb et al. show that if these invariants hold then  $\mathcal{F}_{top}$  is a spanning forest of  $\mathcal{G}$  w.h.p. They also provide an update algorithm which maintains these invariants w.h.p., and takes  $O(\log^4 V)$  time. Consequently, queries are correct w.h.p. across polynomially many updates.

### 3 Hybrid Streaming Connectivity

As a first step towards our main result of a hybrid dynamic connected components algorithm, we present a new semi-streaming algorithm for connected components that uses asymptotically less space than any existing algorithm. We begin by carefully stating the nature of the space advantage. Among the existing algorithms for streaming connectivity, there is no clear winner in terms of asymptotic space usage. This is because some lossless algorithms achieve an  $O(V + E)$  space bound, which is optimal for sparse graphs, while some sketching algorithms achieve a  $O(V \log^2 V)$  space bound which is optimal for dense graphs. The lossless algorithms are suboptimal on dense graphs, and similarly sketch algorithms are suboptimal on sparse graphs.

In contrast, we present a semi-streaming algorithm for connected components that simultaneously matches the state-of-the-art space usage for sparse graphs and dense graphs. In fact in a certain density range, our algorithm uses asymptotically less memory usage than all prior algorithms. Specifically, when the input graph has  $\Omega(V^{1+\epsilon})$  edges for  $0 < \epsilon \leq 1$ , our algorithm uses  $O(V \log^2 V)$  words of space, matching the space of Ahn et al. [3] (sketching approach). When the input graph has  $O(V \log V \log \log V)$  edges, our algorithm uses the same space as an adjacency list representation (lossless approach):  $O(V + E)$ . When the input graph has  $\omega(V \log V \log \log V)$  edges, our algorithm uses asymptotically less space than both Ahn et al. and an adjacency list representation. Theorem 1.2 summarizes our result. Figure 1 depicts the relative asymptotic space usage of a lossless approach, a sketching approach, and our new hybrid approach.

**THEOREM 1.2 (HYBRID STREAMING CONNECTIVITY).** *There exists a semi-streaming algorithm for connectivity on polynomial-length input streams of edge updates with space complexity*

$$O\left(\min\left\{V + E, V \log(V) \log\left(2 + \frac{E}{V}\right)\right\}\right) \text{ w.h.p.}$$

*The update complexity is amortized  $\Theta(\log V)$  w.h.p. The query complexity is worst-case  $\Theta(V \log^2 V)$ .*

**Hybrid Streaming Connectivity Overview.** The main idea behind this result is to lossily store the incidence lists of high-degree vertices

as  $\ell_0$  sketches, while storing the incidence lists of low-degree vertices losslessly (as an explicit set of neighboring vertices). Specifically, we set a fixed **density threshold**  $\delta$ , and we call any vertex  $u$  whose degree is higher than  $\delta$  **heavy**, and otherwise call  $u$  **light**. For each heavy vertex, we maintain an  $\ell_0$  sketch of the characteristic vector (see Section 2.2) of its incidence list, and for each light vertex we store its incidence list losslessly (as an explicit set). As we will see, setting  $\delta$  appropriately minimizes the total size of the data structure, which results in Theorem 1.2. To make this idea work, we require two sketching primitives (one existing and one new) which we now describe.

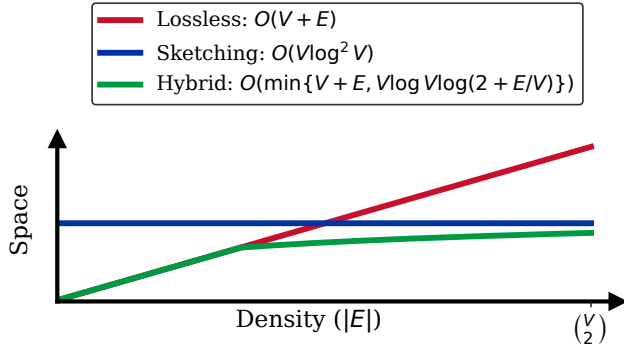
First, note that as the input stream progresses, vertices that were once light may become heavy and vice versa. In particular, when the degree of a heavy vertex  $u$  decreases below  $\delta$ , its edges must be stored losslessly instead of in sketch form. We must therefore **recover**  $\delta$  edges from the sketch of  $u$ , but existing  $\ell_0$  sketching algorithms only guarantee recovery of  $\Theta(\log V)$  edges with high probability. To address this problem, we also maintain an **invertible bloom lookup table (IBLT)** [31] over the set of neighbors for each heavy vertex. An IBLT is a compact data structure representing a set of elements that ensures the successful recovery of all elements in the set with high probability, provided the size of the set is currently below a fixed threshold  $r$ . Regardless of the size of the represented set, the IBLT always uses  $O(r)$  space. Specifically, we use a recent tunable version of IBLT [9] and configure it to be successful with high probability in  $V$ . This results in  $O(r)$  space usage,  $O(\log^2 V)$  insertion time, and  $O(r)$  time to recover all elements (decode).

Second, we present (in Section 3.1) a more compact  $\ell_0$ -sampling primitive, which we call **BALLOONSKETCH**. When the vertex being sketched has high degree, BALLOONSKETCH’s space matches that of existing  $\ell_0$  samplers with constant success probability:  $O(\log V)$ . But unlike existing  $\ell_0$  sampling algorithms, the size of BALLOONSKETCH is  $O(\log(d))$  in expectation where  $d$  is the degree. The key insight is that when the degree is low, in expectation much of the traditional  $\ell_0$  sampler data structure is empty. BALLOONSKETCH exploits this fact to minimize space via aggressive dynamic resizing. Since each vertex has  $O(\log V)$   $\ell_0$  samplers, we are then able to show that the total space across all the  $O(V \log V)$  samplers is  $O(V \log V \log(2 + E/V))$  w.h.p.

Finally, in Section 3.2 we combine the ideas of a density threshold  $\delta$ , IBLT, and BALLOONSKETCH to prove Theorem 1.2. We show that setting the density threshold  $\delta$  to  $\Theta(\log V \log \log V)$  results in the space bound from Theorem 1.2. This value of  $\delta$  is essentially the minimum degree at which a vertex can use asymptotically less space storing its incidence list in BALLOONSKETCH form (i.e.  $\log V \log d \leq d$  when  $d = \Omega(\log V \log \log V)$ ). To facilitate efficient updates while maintaining the space bound, we only recover the lossless incidence list of a vertex that became light when the degree becomes  $\leq \delta/2$ . When a vertex becomes heavy (degree  $> \delta$ ), we convert its lossless incidence list to a BALLOONSKETCH by initiating a new sketch and updating it  $\delta + 1$  times, and also instantiate an IBLT for the vertex.

#### 3.1 $\ell_0$ -Sampling and BALLOONSKETCH

This section introduces BALLOONSKETCH, a new sketching primitive for  $\ell_0$  sampling that uses less space on sparse vectors than prior methods. First we summarize CAMEOSKETCH [56], the prior state-of-the-art algorithm for  $\ell_0$  sampling, and introduce several important definitions that will be used in our definition of BALLOONSKETCH.



**Figure 1: A plot visualizing the space complexity of various graph streaming approaches. Each function is plotted with a fixed  $|V| = 250$ . The x-axis varies the value of  $|E|$ .**

**Prior Work and Definitions for  $\ell_0$ -Sampling.** The core goal of the CAMEOSKETCH algorithm is to act as an  $\ell_0$ -sampler over vectors in  $\mathbb{F}_2^n$ . It achieves this by geometrically distributing the coordinates of an input vector across a series of buckets, ensuring a high probability of isolating a single non-zero coordinate.

A CAMEOSKETCH  $C_h$  is a sequence of  $\rho = \Theta(\log n)$  *sketch buckets*  $B = [b_0, \dots, b_{\rho-1}]$ . Each coordinate  $v \in [n]$  is assigned to exactly two of these buckets: it is always placed in  $b_0$ , designated as the **deterministic bucket**, and it is also placed in a second **random bucket**  $b_i$  chosen geometrically at random. This second assignment is driven by a  $\Theta(1)$ -wise independent hash function  $h : [n] \rightarrow [2^\rho]$  (drawn from a family  $\mathcal{H}$ ); specifically,  $v$  is assigned to bucket  $i = \max\{d \in \mathbb{N} \mid h(v) \equiv 0 \pmod{2^d}\}$ . This explicitly simulates a coin-flipping process, ensuring the probability of landing in  $b_i$  is  $\Pr[v \in b_i] = 1/2^i$ .

Each bucket  $b_i$  maintains a lossy representation of the coordinates assigned to it. It stores two integer values,  $b_i.\text{alpha}$  and  $b_i.\text{gamma}$ , both taking  $\Theta(\log n)$  bits and initialized to 0. When a coordinate  $j$  of the input vector  $x$  is updated, we compute  $h(j)$  to find its assigned bucket  $b_i$ , and perform the following bitwise XOR operations:

- $b_i.\text{alpha} \leftarrow b_i.\text{alpha} \oplus j$
- $b_i.\text{gamma} \leftarrow b_i.\text{gamma} \oplus \gamma(j)$  (where  $\gamma$  is a 2-wise independent hash function)

Maintaining the buckets this way grants us these two properties:

- (1) w.h.p.  $b_i.\text{alpha} = 0$  and  $b_i.\text{gamma} = 0$  if and only if the bucket contains no non-zero coordinates.
- (2) w.h.p.  $b_i.\text{gamma} = \gamma(b_i.\text{alpha})$  if and only if the bucket contains exactly 1 non-zero coordinate. In this case,  $b_i.\text{alpha}$  contains the exact index of that coordinate.

For use in our description of BALLOONSKETCH, we define the following additional properties for vector  $x \in \mathbb{F}_2^n$  with support size  $m$ :

- **Bucket States:** A bucket  $b_i$  is **empty** if its support size is 0. It is **good** if its support size is exactly 1, and **bad** otherwise.
- **Bucket Depth:** We refer to the index of a bucket as its depth.
- **Sketch Depth:** The smallest index  $k \in [\rho]$  such that  $b_i$  is empty for all  $i \geq k$ , or  $\rho$  if no such  $k$  exists.

- **Residual Depth:** Let  $w := \lceil \log_2(m) \rceil$  be the index of the bucket where we mathematically expect exactly one non-zero coordinate to land. The **residual depth** of a sketch is its actual depth minus this expected depth  $w$  (or 0 if the actual depth is  $< w$ ).
- **Sketch Matrix:** A sequence of  $L$  sketches  $(C_{h_1}, \dots, C_{h_L})$  with independent hash functions. By utilizing multiple independent sketches, the matrix boosts the success probability. With  $L = \Omega(\log n)$ , the matrix yields  $\Theta(L)$  successful samples w.h.p. in  $n$ .

**BALLOONSKETCH.** Now we present our new sketching primitive for  $\ell_0$ -sampling, BALLOONSKETCH. The key benefit of BALLOONSKETCH is that it has an asymptotic space advantage over CAMEOSKETCH and all existing  $\ell_0$ -samplers on sparse vectors. On dense vectors, it matches the asymptotic space complexity of CAMEOSKETCH. In a sketch matrix with large enough size, updates to BALLOONSKETCH are as efficient as updates to CAMEOSKETCH. The sampling success probability and the cost of sampling for a BALLOONSKETCH are identical to CAMEOSKETCH. Definition 3.1 defines a BALLOONSKETCH:

**DEFINITION 3.1 (BALLOONSKETCH).** A **BALLOONSKETCH** is a sequence of sketch buckets stored in a dynamically allocated container. The size of the container is always resized to the sketch depth.

Unlike CAMEOSKETCH which statically allocates  $\Theta(\log n)$  buckets, BALLOONSKETCH avoids storing the empty buckets deeper than its sketch depth. Its space usage is therefore proportional to its depth. The procedures for assigning coordinates to buckets and updating the BALLOONSKETCH are almost identical to those of CAMEOSKETCH. However, if an update changes the depth of the sketch, then the bucket list must shrink or grow. The pseudo-codes for the BALLOONSKETCH algorithms are given in Algorithm 1.

---

#### Algorithm 1 BALLOONSKETCH Algorithms

---

```

1: function INITIALIZE( $n$ )
2:    $B \leftarrow []$ ,  $\rho \leftarrow \lg(n) + \Theta(1)$ 
3:   Let  $h : \langle n \rangle \rightarrow [2^\rho]$  be drawn from a family of  $\Theta(1)$ -wise
   independent hash functions.

4: function SAMPLE
5:   if  $B == []$  then return "empty"
6:   for  $i = B.\text{size}$  to 0 do
7:     if  $B[i]$  is good then return ("good",  $B[i].\text{value}$ )
8:   return "fail"

9: function REALLOCATE( $\ell$ )
10:  Initialize new bucket list  $B'$  with  $\ell$  empty buckets.
11:  Copy all non-empty buckets from  $B$  to  $B'$ .
12:   $B \leftarrow B'$ 

13: function UPDATE( $u$ )
14:   $i \leftarrow$  index of random bucket for  $u$  (found from  $h(u)$ )
15:  if  $i \geq B.\text{size}()$  then REALLOCATE( $i + 1$ )
16:  Update bucket  $B[i]$  with  $u$ .
17:  Update bucket  $B[0]$  with  $u$ .
18:  if  $i == B.\text{size}() - 1$  and  $B[i]$  is empty then
19:     $d \leftarrow$  index of deepest non-empty bucket in  $B$ .
20:    if  $d < B.\text{size}() - 1$  then
21:      REALLOCATE( $d + 1$ )

```

---

The update function first computes the random bucket index for  $u$ , and if necessary reallocates the bucket list so that there is a bucket allocated at that index. Then both the random bucket and the deterministic bucket are updated. Finally, if the random bucket was the last bucket in the list, and it became empty from this update, the bucket list is reallocated to remove all empty buckets at the tail. This only has to occur if the random bucket was the last bucket, because otherwise there must be a later bucket that is still non-empty.

Because of the possible need to reallocate the bucket list during an update and the cost of possibly searching for the deepest non-empty bucket, the time for an update to coordinate  $u$  can be proportional to the depth of the random bucket for  $u$ . Since this bucket is chosen randomly between index 0 and  $\rho - 1$ , the worst-case update time for a BALLOONSKETCH is  $\Theta(\log(n))$ . This is notably worse than CAMEOSKETCH's worst-case update time of  $\Theta(1)$ . For similar reasons, the only worst-case space guarantee for BALLOONSKETCH is  $\Theta(\log(n))$  which matches CAMEOSKETCH but is no better for sparse vectors. However, there are three key observations that give hope for asymptotic space savings and update efficiency for a sketch matrix:

- (1) The index of the random bucket a coordinate is assigned to is a geometric random variable, and thus has  $\Theta(1)$  expected value.
- (2) The residual depth of a sketch is bounded by a geometric random variable and thus has  $\Theta(1)$  expected value.
- (3) In a sketch matrix, these random variables for each sketch are independent, and sums of independent random variables concentrate around the sum of their expectations.

These observations guide our proof of Theorem 1.3 which provides high probability bounds on the space usage and update time of a BALLOONSKETCH matrix with  $L$  sketches. When  $L = \Theta(\log n)$  (as is the case for applications like streaming connectivity), The update time for a BALLOONSKETCH matrix is  $O(\log n)$  w.h.p., whereas the update time for a CAMEOSKETCH matrix is  $O(\log n)$  worst-case. In this case the BALLOONSKETCH uses  $O(\log n \log m)$  space w.h.p. (where  $m$  is the number of non-zero elements in the vector), whereas CAMEOSKETCH uses  $\Theta(\log^2 n)$  worst-case space. This indicates that on sparse vectors (low  $m$ ) BALLOONSKETCH can use asymptotically less space than CAMEOSKETCH. The full proof is provided in Appendix A.

**THEOREM 1.3 (BALLOONSKETCH).** *There exists an  $\ell_0$ -sampler, BALLOONSKETCH, such that given  $L$  independent BALLOONSKETCHES, each sketching the same vector  $x \in \mathbb{R}^n$  with  $m = \|x\|_0$ :*

- (1) *The total space usage of the sketches is  $O(L \log m + \log n)$  w.h.p.*
- (2) *The total cost of one update to each sketch is  $O(L + \log n)$  w.h.p.*

### 3.2 Proof of Theorem 1.2

Our hybrid streaming algorithm generally maintains the adjacency lists of light vertices as an explicit set, and maintains the adjacency lists of heavy vertices as a BALLOONSKETCH matrix with  $L = \Theta(\log V)$  and an IBLT with recovery size  $\delta/2$ . For the explicit set, we will use a standard balanced binary search tree (BBST) data structure. As mentioned previously, we only convert a vertex's adjacency list from a sketch to an explicit set if its degree becomes  $\leq \delta/2$ . This means there may be some light vertices that are stored as a sketch, but their degree is  $\Omega(\delta)$ . To simplify things, we say a vertex whose adjacency list is stored as a sketch is in **sketch form**, and we say it is in **explicit form** otherwise. A vertex in sketch form has  $d(v) = \Omega(\delta)$  and a vertex in explicit form has  $d(v) = O(\delta)$ .

One additional detail in our algorithm is that for edges with at least one endpoint in explicit form, we will only store the edge in the adjacency list of one of its endpoints: the one that is in explicit form or either endpoint if both are in explicit form. All other edges between two vertices in sketch form are stored by both endpoints. This not only saves a bit of space in practice, but it is a crucial invariant to ensure the success of our connected components query algorithm (described in Appendix B). When analyzing the space usage, we make the simplifying assumption that each vertex stores all of its incident edges, which serves as an upper bound for the true space usage.

In the remainder of this section we discuss and analyze the space usage, the update algorithm, and the connected component query algorithm. Together these things compose the proof of Theorem 1.2.

**Space Usage.** For a single vertex of degree  $d$ , a set representation (BBST) requires  $\Theta(d)$  space. In contrast, the BALLOONSKETCH matrix representation consumes  $O(\log V \log d)$  space w.h.p. The optimal density threshold  $\delta$ , which dictates whether the sketch or the set is more space-efficient, occurs where these two complexities are equal. Solving for  $d$  yields a threshold of  $\delta = \Theta(\log V \log \log V)$ . At this density, the space complexity of the BALLOONSKETCH matrix evaluates to  $O(\log V \log(\log V \log \log V)) = O(\log V \log \log V)$  w.h.p.

We prove in Appendix D that by setting the density threshold  $\delta = \Theta(\log V \log \log V)$ , the total space complexity across all vertices satisfies both  $O(V + E)$  and  $O(V \log V \log(2 + E/V))$  w.h.p. These bounds collectively establish the space requirements for Theorem 1.1.

**Stream Updates.** First we describe edge updates assuming no promotion or demotion occurs. If we insert or delete an edge  $(u, v)$  and both endpoints are in sketch form, we update the BALLOONSKETCH matrix and IBLT for both endpoints. If either endpoint is in explicit form, the edge should only be stored in the adjacency list of one of the endpoints that is in explicit form. For insertions, we arbitrarily pick one of these endpoints and insert the edge into its set. For deletions, we check both endpoints and delete the edge if the set contains it.

When an edge insertion causes a vertex  $u$  in explicit form to exceed the density threshold ( $d(u) > \delta$ ),  $u$  is promoted to sketch form. At this point,  $u$  converts its set into sketch form by initiating a new BALLOONSKETCH and an IBLT. To maintain the invariant,  $u$  iterates through the edges currently in its set and redistributes them as follows. For neighbors in sketch form, these edges now exist between two vertices in sketch form. Vertex  $u$  adds the edge to its new sketch and issues an update to the neighbor  $v$  instructing it to also add the edge to its sketch. For neighbors in explicit form, these edges now have exactly one endpoint in explicit form (the neighbor). Since the vertex  $u$  is now in sketch form, it can no longer store these edges. It therefore pushes these edges to the respective neighbors' sets and does not add them to its own sketch.

Conversely, if edge deletions cause a sketch-form vertex  $u$  to drop to  $d_u \leq \delta/2$ , it demotes to explicit form. Because the invariant ensures  $u$ 's IBLT only tracks edges where both endpoints were in sketch form,  $u$  uses its IBLT to perfectly recover these edges. It moves these recovered edges into its new explicit set and issues updates to those neighbors (which are still in sketch form) to delete the edges from their sketches. Crucially, any edges  $u$  shares with other explicit form vertices were already uniquely stored in the neighbors' sets. We prove in Appendix B that updates have amortized cost  $O(\log V)$  w.h.p.

## 4 Hybrid Fully-Dynamic Connectivity

We prove Theorem 1.1, resulting in a hybrid fully-dynamic graph connectivity algorithm.

**THEOREM 1.1 (HYBRID DYNAMIC CONNECTIVITY).** *There exists a dynamic connectivity algorithm with space complexity*

$$O\left(\min\left\{V + E, V \log(V) \log\left(2 + \frac{E}{V}\right)\right\}\right) \text{ w.h.p.}$$

*The update complexity is amortized  $O(\log^4 V)$ . The query complexity is worst-case  $O(\log V / \log \log V)$ .*

The first component of this is a new purely sketch-based dynamic connectivity algorithm with space complexity  $O(V \log V \log(2 + E/V))$  w.h.p. This results from combining our BALLOONSKETCH techniques described in Section 3 with the sketch-based dynamic connectivity algorithm of Gibb et al. [30] (Gibb’s algorithm).

Unlike the streaming algorithm from Section 3, it is not as straightforward to develop a hybrid version of this dynamic algorithm, because the data structures representing the neighborhood of each vertex are not neatly separated: they are heavily intertwined in the cutset data structures which maintain aggregate sketches over several subsets of vertices. Thus we can not simply use a hybrid approach on the granularity of each vertex.

Instead, to prove Theorem 1.1 we introduce a novel dynamic connectivity framework which uses different data structures to store the **sparse regions** and **dense regions** of the input graph. Employing a hybrid approach over *regions* of the graph allows us to overcome the difficulty of doing it on the granularity of vertices. Our framework is general, and can use any two dynamic connectivity algorithms (assuming they support a certain reasonable set of operations) to store the sparse and dense regions of the graph. Our framework is intended to use a **lossless algorithm** to store the sparse regions, and a **sketch-based algorithm** to store the dense regions. We show that by using our new BALLOONSKETCH-based algorithm as the dense algorithm and using a lossless algorithm with linear space (and low polylogarithmic time updates), such as the Cluster Forest algorithm [21, 59], as the sparse algorithm, Theorem 1.1 is achieved.

### 4.1 BALLOONSKETCH-Based Dynamic Algorithm

The starting point for new sketch-based algorithm is Gibb’s algorithm [30] (summarized in Section 2.3). Rather than using the  $\ell_0$ -sketch algorithm of Cormode et al. [18], we will instead use our new  $\ell_0$ -sketch from Section 3.1, BALLOONSKETCH. Recall that the data structures maintained by Gibb’s algorithm are  $\Theta(\log V)$  levels of cutset data structures (see Section 2.3). Using the sketch of Cormode et al., the space used by each cutset is  $O(V \log V)$  w.h.p., resulting in  $O(V \log^2 V)$  space w.h.p. overall.

We prove that by using BALLOONSKETCH instead of the sketch of Cormode et al. in the cutset data structures, the aggregate space usage of each cutset data structure can be reduced to  $O(V \log(E/V))$  in expectation, and the total space usage for the entire algorithm can be reduced to  $O(V \log(V) \log(E/V))$  w.h.p.

There is inherent flexibility in the specific implementation of the cutset data structure, as any dynamic tree supporting subtree aggregate queries can be used. Previous works such as Gibb’s algorithm and CUPCaKE use Euler-tour trees (ETTs) for this purpose, but the asymptotic space usage is the same regardless of the choice of dynamic

tree. In this work, we implement the cutset data structures using UFO trees [23] augmented with BALLOONSKETCH sketches. We chose to use UFO trees because they simplify the analysis of the total space complexity across all BALLOONSKETCHES in our data structures.

Our overall dynamic connectivity data structure and algorithms remain equivalent to Gibb’s algorithm. Since UFO trees provide the same update complexity as ETTs, the overall update and query complexity of our dynamic connectivity algorithm remain the same:  $O(\log^4 V)$  and  $O(\log V / \log \log V)$  respectively. Lemma 4.1 summarizes the results of our algorithm.

**LEMMA 4.1.** *There exists a dynamic connectivity algorithm with space complexity  $O(V \log V \log(2 + E/V))$  w.h.p. The update complexity is  $O(\log^4 V)$ . The query complexity is  $O(\log V / \log \log V)$ .*

**Space Usage.** Here we provide an overview of the space complexity analysis for our new algorithm. We provide the full proof of Lemma 4.1 in Appendix C. In our prior proof of the space usage of our streaming algorithm (Theorem 1.2, Section 3), we established that the total space usage of the BALLOONSKETCH matrix for each vertex  $v$  is  $\Theta(\log V \log d_v)$  w.h.p. Proving the space bounds for our dynamic algorithm introduces two main challenges.

First, we must account for space in internal UFO tree nodes, not just the leaves representing the vertices of the graph. While leaf nodes asymptotically dominate the total node count, higher-level internal nodes may sketch an increasing average number of edges. We must rigorously prove that the sum of these potentially larger internal BALLOONSKETCHES does not asymptotically exceed the leaf-level space.

Second, the  $\Theta(\log V)$  sketches for a vertex are distributed across independent UFO trees. Naively bounding the space of each sketch individually yields a suboptimal  $O(V \log^2 V)$  total space w.h.p. Instead, we must leverage the independent random seeds across tiers to aggregate sketch residual depths and apply concentration bounds, achieving the tighter  $O(V \log V \log(2 + E/V))$  bound w.h.p.

**Vertex Updates.** For use in our hybrid dynamic connectivity algorithm described in the next section, we describe how to modify our new BALLOONSKETCH-based dynamic connectivity algorithm to support a changing set of vertices. Specifically we support the operations  $\text{INSERTVERTEX}(v)$  and  $\text{DELETEVERTEX}(v)$  which insert or delete a degree 0 vertex in the graph.

Let  $V_D$  be the current number of vertices in the graph. We assume that there are at most  $V$  total vertices at any time, that is  $V_D \leq V$ . To maintain that our algorithm is correct w.h.p. in  $V$  we still maintain  $\Theta(\log V)$  tiers at all times, even if the actual number of vertices  $V_D$  is actually much less than  $V$ . Consequently, the space usage of the data structure is  $O(V_D \log V \log(2 + E/V_D))$  with high probability in  $V$  (note that the first logarithmic term is in terms of  $V$ , not  $V_D$ ).

To support vertex updates, we maintain a hash map that maps each vertex  $v$  to the UFO tree leaf cluster representing them in each tier. Vertex insertions allocate a new UFO cluster with a BALLOONSKETCH in each tier, and vertex deletions free them all. Both take  $O(\log V)$  time, since an empty BALLOONSKETCH has constant size.

### 4.2 Hybrid Dynamic Connectivity Framework

In this section, we introduce our hybrid dynamic connectivity framework. First we provide a few key definitions. As before, our framework is parametrized by some fixed density threshold  $\delta$ .



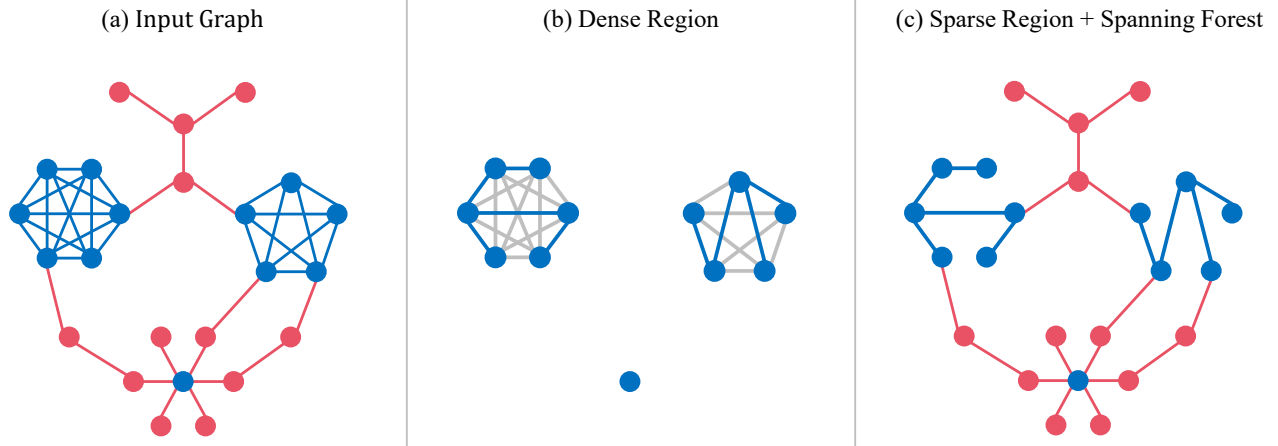


Figure 2: An example of the partitioning of a graph by our hybrid framework with density threshold  $\delta = 3$ . Subfigure (a) shows a graph  $\mathcal{G}$ . The blue vertices and edges in (a) represent heavy vertices and dense edges, and the red vertices and edges represent light vertices and sparse edges. Subfigure (b) shows the dense region of  $\mathcal{G}$  which is stored by  $ALG_D$ , indicating the spanning forest with blue edges. Subfigure (c) shows the union of the sparse region of  $\mathcal{G}$  and the spanning forest from  $ALG_D$ , which is stored by  $ALG_S$ .

- A **heavy vertex** is defined as a vertex  $v$  such that  $d(v) \geq \delta$ .
- A **light vertex** is defined as a vertex  $v$  such that  $d(v) < \delta$ .
- A **dense edge** is defined as an edge  $e = (u, v)$  such that  $u$  and  $v$  are both heavy vertices.
- A **sparse edge** is defined as any edge that is not a dense edge.
- The **dense region** of  $\mathcal{G}$  is the subgraph induced by the heavy vertices. That is all the heavy vertices, and all the dense edges (between two heavy vertices).
- The **sparse region** of  $\mathcal{G}$  contains all vertices in  $\mathcal{V}$  and all the sparse edges.

Note that by these definitions, the edges of the graph are partitioned between the dense region and sparse region, while there may be overlap in the sets of vertices in each region (the sparse region always has all vertices and the dense region has a subset). Our framework uses two separate dynamic connectivity algorithms to store the dense and sparse regions of the graph:

- **Dense Algorithm** ( $ALG_D$ ): stores only the dense region of  $\mathcal{G}$ .
- **Sparse Algorithm** ( $ALG_S$ ): stores the union of the sparse region of  $\mathcal{G}$  and a spanning forest of the dense region.

Specifically, we require that  $ALG_D$  explicitly stores a spanning forest, and that  $ALG_S$  contains the same spanning forest of the dense region that is maintained by  $ALG_D$ . This means that all edges are partitioned between  $ALG_D$  and  $ALG_S$  except for these **dense spanning forest edges** which are stored by both algorithms. Figure 2 shows an example of a graph  $\mathcal{G}$ , its dense and sparse regions, and the subgraphs maintained by  $ALG_D$  and  $ALG_S$ .

Our framework is designed to use a lossless algorithm using  $O(V + E)$  space as  $ALG_S$ , and a sketch-based algorithm using at most  $O(V \log^2 V)$  space as  $ALG_D$ , however this is not a requirement. Our framework only requires that (in addition to supporting all the operations required by the dynamic connectivity problem)  $ALG_D$  and  $ALG_S$  have the following properties, which are standard or easily supported by most existing dynamic connectivity algorithms:

Requirements for  $ALG_D$ :

- Explicitly maintains a **spanning forest**  $ALG_D.\mathcal{F}$  over its graph. Note that some dynamic connectivity algorithms can answer connectivity queries, but do not maintain a spanning forest.
- **INSERTEDGE** and **DELETEEDGE** operations **return a list of the changes to  $ALG_D.\mathcal{F}$**  induced by the update.
- **INSERTVERTEX( $v$ )/DELETEVERTEX( $v$ )**: Given a vertex  $v$  with  $d(v) = 0$ , inserts or deletes  $v$  in the graph. This is necessary since the vertex set in the dense region of the graph may change.

Requirements for  $ALG_S$ :

- **INCIDENTEDGES( $v$ )**: Return all edges incident to vertex  $v$ . This is not supported by existing sketch-based methods, but is simple for most lossless methods.
- **$e \in ALG_S.\mathcal{E}$** : Returns whether edge  $e$  is in the graph maintained by  $ALG_S$ . This is also not possible for existing sketch-based methods, but easy for lossless methods.

Most notably,  $ALG_D$  require each update to return a list of changes to its spanning forest. We require this because different dynamic connectivity algorithms can vary greatly in how they update their spanning forest in response to edge insertions or deletions. For example, any insertions or deletions in sketch-based algorithms like Gibb's algorithm can induce up to  $\Theta(\log V)$  spanning forest changes.

In addition to  $ALG_D$  and  $ALG_S$ , our framework stores a few extra data structures to interface between the two. To handle vertices that change from heavy to light, our framework maintains an IBLT with recovery size  $\delta/2$ , for each vertex in  $ALG_D$ . We also store an array of  $V$  boolean values indicating whether each vertex is in  $ALG_D$ . The last piece of additional data stored by our framework is the **global degree**  $d(v)$  for each vertex  $v$  (its degree in the input graph  $\mathcal{G}$ ). This is necessary because the edges of  $\mathcal{G}$  may be divided between  $ALG_D$  and  $ALG_S$  and some edges may be in both, so it is hard to discern the total degree using just these data structures.

In the remainder of this section we abstractly describe the query and update algorithms for our hybrid framework in terms of  $ALG_S$



and  $ALG_D$ . In the next section (Section 4.3) we analyze the theoretical guarantees provided by our framework when using the Cluster Forest algorithm [59] as  $ALG_S$ , and the BALLOONSKETCH-based algorithm from Section 4.1 as  $ALG_D$ .

**Query Algorithm.** Since  $ALG_S$  stores the union of the sparse region of the graph and a spanning forest of the dense region, all vertices in  $\mathcal{G}$  are in  $ALG_S$ , and the connected components in  $ALG_S$  are exactly the same as the connected components of  $\mathcal{G}$ . Thus, a query  $\text{CONNECTED}(u, v)$  in our framework can be answered at any time by calling  $ALG_S.\text{CONNECTED}(u, v)$ .

---

**Algorithm 2** Edge insertion algorithms for the hybrid framework.

---

```

1: function INSERTEDGE( $e = \{u, v\}$ )
2:   for  $x \in \{u, v\}$  do
3:      $d(x) \leftarrow d(x) + 1$ 
4:     if  $x \notin ALG_D.\mathcal{V}$  and  $d(x) > \delta$  then
5:       PROMOTEVERTEX( $x$ )
6:   if  $u \in ALG_D.\mathcal{V}$  and  $v \in ALG_D.\mathcal{V}$  then
7:      $ALG_D.\text{INSERTEDGE}(e)$ 
8:      $\text{IBLT}_u.\text{INSERT}(v), \text{IBLT}_v.\text{INSERT}(u)$ 
9:     Update  $ALG_S$  with all changes to  $ALG_D.\mathcal{F}$ .
10:  else
11:     $ALG_S.\text{INSERTEDGE}(e)$ 

12: function PROMOTEVERTEX( $v$ )
13:    $ALG_D.\text{INSERTVERTEX}(v)$ 
14:   Instantiate  $\text{IBLT}_v$ 
15:   for  $e = \{v, u\} \in ALG_S.\text{INCIDENTEDGES}(v)$  do
16:     if  $u \in ALG_D.\mathcal{V}$  then
17:        $ALG_D.\text{INSERTEDGE}(e)$ 
18:        $\text{IBLT}_u.\text{INSERT}(v), \text{IBLT}_v.\text{INSERT}(u)$ 
19:      $ALG_S.\text{DELETEEDGE}(e)$ 
20:   Update  $ALG_S$  with all changes to  $ALG_D.\mathcal{F}$ .

```

---

**Insertion Algorithm.** Algorithm 2 shows the pseudo-code for inserting an edge  $e = \{u, v\}$  in our hybrid framework. First, the algorithm updates the global degree of  $u$  and  $v$ , and if their new degree is  $> \delta$ , calls PROMOTEVERTEX on them (lines 2–5). PROMOTEVERTEX( $v$ ) first inserts  $v$  into  $ALG_D$  and instantiates an IBLT for  $v$  (lines 13–14). Next, for all the edges incident to  $v$  in  $ALG_S$ , if the other endpoint is also in  $ALG_D$  they are inserted into  $ALG_D$  and the IBLTs, and deleted from  $ALG_S$  (lines 15–19). This ensures that  $ALG_D$  includes all edges that became dense edges when  $v$  became a heavy vertex. Finally, we must update  $ALG_S$  to contain the spanning forest maintained by  $ALG_D$ . As mentioned previously, our framework requires that edge insertion operations in  $ALG_D$  report a list of changes to the spanning forest caused by these insertions. Our algorithm uses this list of changes to insert or delete any necessary edges in  $ALG_S$  so that it always stores the exact spanning forest maintained by  $ALG_D$ .

Now that the algorithm has dealt with vertices becoming heavy, the next step is to actually insert the new edge  $e$ . If both endpoints of  $e$  are in the dense region, we insert  $e$  into  $ALG_D$ , update the IBLTs for both endpoints, and again update  $ALG_S$  with all the spanning forest changes in  $ALG_D$  (lines 6–9). Else we simply insert  $e$  into  $ALG_S$ , since the edge is in the sparse region of the graph (line 11).

---

**Algorithm 3** Edge deletion algorithms for the hybrid framework.

---

```

1: function DELETEEDGE( $e = \{u, v\}$ )
2:   for  $x \in \{u, v\}$  do
3:      $d(x) \leftarrow d(x) - 1$ 
4:     if  $x \in ALG_D.\mathcal{V}$  and  $d(x) \leq \delta/2$  then
5:       DEMOTEVERTEX( $x$ )
6:   if  $u \in ALG_D.\mathcal{V}$  and  $v \in ALG_D.\mathcal{V}$  then
7:      $ALG_D.\text{DELETEEDGE}(e)$ 
8:      $\text{IBLT}_u.\text{DELETE}(v), \text{IBLT}_v.\text{DELETE}(u)$ 
9:     Update  $ALG_S$  with all changes to  $ALG_D.\mathcal{F}$ .
10:  else
11:     $ALG_S.\text{DELETEEDGE}(e)$ 

12: function DEMOTEVERTEX( $v$ )
13:   for  $e = (u, v) \in \text{IBLT}_v.\text{RECOVER}()$  do
14:      $ALG_D.\text{DELETEEDGE}(e)$ 
15:      $\text{IBLT}_u.\text{DELETE}(v)$ 
16:     if  $e \notin ALG_S.\mathcal{E}$  then
17:        $ALG_S.\text{INSERTEDGE}(e)$ 
18:   Update  $ALG_S$  with all changes to  $ALG_D.\mathcal{F}$ .
19:    $ALG_D.\text{DELETEVERTEX}(v)$ 
20:   Delete  $\text{IBLT}_v$ 

```

---

**Deletion Algorithm.** Algorithm 3 shows the pseudo-code for deleting an edge  $e = \{u, v\}$  in our hybrid framework. First, the algorithm updates the global degree of  $u$  and  $v$ , and if their new degree is  $\leq \delta/2$ , calls DEMOTEVERTEX on them (lines 2–5). Similar to our hybrid streaming algorithm (Section 3), we only demote vertices when their degree drops down to  $\delta/2$  to prevent thrashing and enable efficient amortized update complexity. DEMOTEVERTEX( $v$ ) uses the IBLT of  $v$  to extract the at most  $\delta/2$  edges incident to  $v$  in  $ALG_D$  (line 13). For each of these edges, they are deleted from  $ALG_D$ , deleted from the IBLT of the other endpoint, and inserted into  $ALG_S$  if they are not already present there (lines 14–17). Once again, any changes caused to the spanning forest maintained by  $ALG_D$  by these deletions must be updated in  $ALG_S$ . In particular, this step must avoid deleting the edges just inserted to  $ALG_S$  in the previous loop. Finally,  $v$  is deleted from  $ALG_D$ , and  $\text{IBLT}_v$  is freed (lines 19–20).

Now the algorithm deletes edge  $e$ . If both endpoints of  $e$  are in the dense region, delete  $e$  from  $ALG_D$ , update the IBLTs for both endpoints, and again update  $ALG_S$  with all the spanning forest changes in  $ALG_D$  (lines 6–9). This last step is particularly important, because if a spanning forest edge was deleted in  $ALG_D$ , this edge would currently be present in  $ALG_S$ , and this step also deletes it there (disconnecting the graph represented by  $ALG_S$ ). If a replacement edge was found in  $ALG_D$ , it becomes part of the spanning forest and is also inserted into  $ALG_S$ . If both endpoints of  $e$  are not in the dense region, we simply delete  $e$  from  $ALG_S$ , since the edge is only in the sparse region of the graph (line 11).

### 4.3 Proof of Theorem 1.1

We prove Theorem 1.1 by analyzing the performance of our hybrid framework using our BALLOONSKETCH-based algorithm from Section 4.1 as  $ALG_D$ , and the Cluster Forest algorithm [59] as  $ALG_S$ . It is

easily verifiable that our BALLOONSKETCH-based algorithm and Cluster Forest meet the requirements for  $ALG_D$  and  $ALG_S$  respectively.

We configure the density threshold to be  $\delta = \log V \log \log V$ . Intuitively, this is the value of  $\delta$  where if every vertex has degree exactly  $\delta$ , then the space usage of a lossless representation is asymptotically equal to the space usage required in our BALLOONSKETCH-based algorithm:  $O(V + V\delta)$  versus  $O(V \log V \log(2 + \delta V/V))$ .

We prove that the space complexity of our algorithm is  $O(V + E)$  w.h.p. and  $O(V \log V \log(2 + E/V))$  w.h.p in Appendix D. The key observation for the first bound is that the space used by  $ALG_D$  is  $O(E)$  since the existence of  $V_D$  vertices in  $ALG_D$  implies the total number of edges  $E$  is at least  $V_D \delta$ . The key observation for the second bound is that  $ALG_S$  uses  $O(V \log V \log(2 + E/V))$  space since every light vertex has maximum degree  $\delta$ , so the number of sparse edges is  $O(V\delta)$ . We also prove in Appendix D that the update and query complexities are amortized  $O(\log^4 V)$  and  $O(\log V / \log \log V)$ .

## 5 System Design / Implementation

In this section, we describe **HYBRIDSCALE** (Hybrid Sketching Connectivity Algorithm with Lossless Edges), our hybrid sketching system for dynamic connectivity based on the algorithm from Section 4. As per the name, our system can seamlessly *scale* existing dynamic connectivity systems to larger and denser real-world graphs. HYBRIDSCALE consists of three major components:

- (1)  $ALG_S$ : the **Cluster Forest implementation** of De Man et al. [21].
- (2)  $ALG_D$ : **BALLOONDC**, an implementation of our new BALLOONSKETCH-based dynamic connectivity algorithm from Section 4.1.
- (3) A **hybrid interface** that manages communication and movement of edges and vertices between the first two components.

We chose to use this implementation of the Cluster Forest because it provides good guarantees on performance and provides robust state-of-art performance regardless of the structure of the input graph. However, the first two components can be easily interchanged with any other existing or future dynamic connectivity systems that meet the basic requirements for  $ALG_S$  and  $ALG_D$ . All of our implementations are written in C++ and compiled with -O3 optimization.

**Parameter Configuration.** A key parameter in our hybrid algorithm is the density threshold  $\delta$ . To achieve good theoretical bounds  $\delta$  must be  $\Theta(\log V \log \log V)$ . In HYBRIDSCALE, we set  $\delta = 25 \cdot \lceil \log_2 V \rceil$ . We chose this threshold because we experimentally found it to be roughly the best value in terms of minimizing the total memory usage. Conceptually, the constant multiple of  $25 \approx 6 \cdot 4$  can be derived from the fact that  $\log \log V$  is at most 6 for all of our inputs, and the space usage of a single sketch bucket is 4 times that of storing a neighbor entry in a lossless neighbor array.

In HYBRIDSCALE, we demote a vertex when its degree drops to  $\delta/8$  (as opposed to  $\delta/2$  in our theoretical analysis). This choice manages a tradeoff between the aggressiveness of vertex reclamation and global memory overhead. While a higher threshold would move vertices back to the space-efficient Cluster Forest sooner, it would also require every vertex in the dense subsystem to maintain a larger IBLT. Provisioning each IBLT for  $\delta/2$  would cause these structures to occupy a significantly larger portion of the system's total memory. By lowering the threshold to  $\delta/8$ , we decrease this memory overhead, at the cost of some vertices remaining in the dense subsystem a bit longer.

A final parameter is the number of tiers used in BALLOONDC, which must be  $\Theta(\log V)$  in theory. The number of tiers impacts the error rate of the Monte Carlo algorithm. We use  $\lceil \log_2 V \rceil$  tiers, which is slightly more aggressive than prior work on graph-sketching methods [56] and below the number of tiers needed according to a standard analysis of the semi-streaming algorithm [3]. We experimentally verify that HYBRIDSCALE actually uses only 64% of these  $\lceil \log_2 V \rceil$  tiers (geometric mean across all tested datasets.) We further note that HYBRIDSCALE consistently uses fewer tiers than CUPCaKE. This is intuitively because our design choice to sketch only the dense regions of the graph makes finding connectivity via Borůvka take significantly fewer rounds. Additionally, while we provision enough tiers for the maximum number of vertices, the actual number of vertices present in the dense region is often much lower, naturally reducing the required number of tiers. Together, these factors indicate that there may be an opportunity for more aggressive space savings by reducing the number of tiers stored.

### 5.1 BALLOONDC

BALLOONDC uses the implementation of CUPCaKE [22] as a starting point, but makes many modifications. The dynamic tree and  $\ell_0$ -sketching primitives we use are both different than that of CUPCaKE, we handle insertion/deletion of vertices which CUPCaKE does not support, and we improve the buffering techniques of CUPCaKE.

**Cutset Implementation.** Systems like CUPCaKE and BALLOONDC rely on multiple tiers of cutset data structures (see Section 2.3 for details). In BALLOONDC, we implement the cutset data structure using link-cut trees [53] based on splay trees [54]. Note that our theoretical analysis uses UFO trees [23] to implement the cutset data structure for simplicity of analysis, but in practice we use link-cut trees for their lower empirical memory footprint. Each vertex in the link-cut tree is augmented with a BALLOONSKETCH, which we develop a new implementation for. Link-cut trees typically do not support subtree queries, but they work well for subtree queries in our use-case since sketch addition is an invertible function, and they only need to store one sketch per vertex (avoiding internal aggregate sketches). Additionally, we optimize sketch update operations in the link-cut trees by only splaying an accessed node if its depth exceeds some constant times  $\log V$ .

**Improved Batching.** The CUPCaKE system was optimized to rapidly process batches of updates that induce no structural changes in its cutsets. To take advantage of this, they speculatively buffer batches of updates in hopes that the entire batch will induce no structural changes. This has two major drawbacks. The first is that if the batch of updates does induce a structural update, the system must undo some of the work it did and process each update in the batch one-by-one without the batch optimizations. The second issue is that the update buffer must be flushed whenever a query occurs, so this optimization can be entirely ineffective in streams with many queries, and it can significantly increase query latencies.

BALLOONDC improves the buffering techniques of CUPCaKE so that any type of updates can be processed in a batch, including ones that induce structural changes. This overcomes the first drawback of CUPCaKE's buffering strategy. Regarding the second drawback, our HYBRIDSCALE uses Cluster Forest to answer queries, so flushing the update buffer immediately in BALLOONDC is not required for queries. The only thing that requires flushing the buffer of updates

is a deletion of a current spanning forest edge in BALLOONDC, which happens infrequently, and does not impact query latency.

## 5.2 Hybrid Interface

The hybrid interface manages communication and movement of edges and vertices between the Cluster Forest and BALLOONDC systems, implementing the abstract logic of the algorithm from Section 4. The set of currently dense vertices is stored in a hash table, and the total degree of each vertex is stored as an integer. The hybrid interface also stores an IBLT for each vertex  $v$  that allows for recovering all of the edges incident to a vertex being demoted from the dense region (see next paragraph for implementation details). To optimize communication, edges that should be moved to the dense subsystem are collected in an update buffer rather than being sent immediately. This buffer is flushed to the dense subsystem in batches either when it reaches capacity or when an incoming update deletes an active edge from the dense spanning forest. Finally, the hybrid interface stores a list of modifications made to the spanning forest during BALLOONDC updates. We appropriately append to this list while processing updates in BALLOONDC, and apply all these changes to the Cluster Forest after finishing the updates.

**Recovery IBLTs.** Our IBLT implementation is based on the original IBLT algorithm [31] and uses CAMEOSKETCH buckets as a building block. Our specific implementation utilizes two tiers of IBLTs with  $k = 3$  hash locations and  $1.3\delta/8$  buckets and  $\max\{\log_2 V, 0.2\delta/8\}$  buckets respectively for the two tiers, using xxHash [17] with iterated seeds to ensure unique hash mapping. Although the original IBLT algorithm’s recovery is only correct w.h.p. in  $\delta$  (not in  $V$ ), we found that the IBLT recovery succeeded in 99.9% of cases in practice. To ensure correctness of our overall system, any failed recovery operation simply aborts the demotion, retaining the vertex in the dense subsystem until a later demotion attempt. While this approach may keep light vertices in the dense subsystem longer, the exceptionally low error rate ensures that any resulting overhead in space or time executing demotions remains negligible.

## 6 Experiments

Each experiment reported in this section consists of running HYBRIDSCALE on an input stream comprised of edge insertions and deletions generated from some static graph (by inserting all the edges then deleting the set of inserted edges). All experiments were run on a 48-core AMD EPYC 7643 CPU with 256GB of RAM.

**Datasets.** Table 1 summarizes the datasets used in our experiments. The social, web, and road graphs are sourced from SuiteSparse [20] and NetworkRepository [49], and are widely used in academic research. The kron graphs are synthetic Kronecker random graphs generated according to the Graph500 specification [7]. They are studied in all prior graph sketching experimental research because they are dense and exhibit structural properties of real-world networks.

The  $knn/rs$  graphs are symmetrized  $k$ -nearest neighbors ( $knn$ ) and range search ( $rs$ ) graphs over a 1M-point subset of the SIFT1B and MSSPACEV1B datasets from BIGANN [52]. We generate these graphs using ParlayANN [62] to compute the ground truth  $knn$  or  $rs$ . For  $knn$  graphs, we use  $k = 500$ . For  $rs$  graphs we used a squared euclidean distance radius of 50K and 10K respectively for SIFT1B and

| Graph       | Type     | $V$   | $E$   | $2E/V$ | Max.<br>$k$ -Core | Cite |
|-------------|----------|-------|-------|--------|-------------------|------|
| Google+     | social   | 108K  | 10.8M | 201    | 676               | [41] |
| Friendster  | social   | 65.6M | 1.81B | 55.1   | 304               | [61] |
| Twitter     | social   | 41.7M | 1.2B  | 57.7   | 2488              | [40] |
| Orkut       | social   | 3.07M | 117M  | 76.3   | 253               | [43] |
| ENWiki      | web      | 4.21M | 91.9M | 43.7   | 145               | [10] |
| Youtube     | web      | 1.16M | 2.99M | 5.16   | 51                | [43] |
| RoadUSA     | road     | 23.9M | 28.9M | 2.41   | 3                 | [48] |
| RoadGermany | road     | 12.3M | 16.1M | 2.63   | 3                 | [48] |
| SIFT-RS50K  | $knn/rs$ | 1M    | 227M  | 454    | 4301              | [52] |
| SIFT-KNN500 | $knn/rs$ | 1M    | 411M  | 821    | 499               | [52] |
| MSSP-RS10K  | $knn/rs$ | 1M    | 547M  | 1090   | 18778             | [52] |
| MSSP-KNN500 | $knn/rs$ | 1M    | 380M  | 760    | 499               | [52] |
| Kron-13     | kron     | 8.19K | 16.7M | 4070   | 2827              | [7]  |
| Kron-15     | kron     | 32.8K | 267M  | 16300  | 11306             | [7]  |
| Kron-16     | kron     | 65.5K | 1.07B | 32700  | 22623             | [7]  |

Table 1: Summary of the graph datasets in our experiments.

MSSPACEV1B. The choice of  $k$  or the search radius when constructing  $knn/rs$  graphs directly controls graph density and is a central modeling question in large-scale graph construction pipelines [13, 24, 62]. Larger values of  $k$  or radius yield denser graphs with richer connectivity, which typically improves the quality of downstream tasks such as graph-based clustering, semi-supervised learning, and retrieval, but substantially increases storage and processing costs. Smaller values yield sparser graphs that are cheaper to process but can under-connect the underlying data [13]. Because of this trade-off, practitioners building  $knn/rs$  graphs must target a range of densities depending on the application, which motivates evaluating HYBRIDSCALE across moderately large  $k$  and radius regimes.

Importantly, our list of datasets includes graphs with similar average densities but very different dense cores (see  $k$ -core column in Table 1 which reports the largest  $k$  such that the graph has a non-empty  $k$ -core). The  $k$ -core of a graph is a popular metric for identifying well-connected communities within a graph; in particular, all vertices in a  $k$ -core have induced degree (i.e., degree restricted only within this subgraph) at least  $k$ . Thus, the presence of a large  $k$ -core in a graph implies the existence of a subgraph where all vertices have high degree (in particular, the average degree is at least  $k$ ). The social network and road graphs both have relatively low average densities, but the social networks have much larger, denser cores than the road networks. Similarly, for the  $rs$  and  $knn$  graphs, average degrees are similar but the  $rs$  graphs have larger, denser cores. This allows us to measure the impact of core size and density on our system performance.

**Comparison Systems.** In these experiments we evaluate the performance of Cluster Forest and CUPCaKE, since they are state-of-the-art implementations of dynamic connectivity systems optimized for sparse and dense graphs respectively. Our focus on these specific systems is intended to demonstrate the efficacy of the hybrid framework. Because the framework is modular, HYBRIDSCALE is designed so that any other sparse or dense optimized implementations developed in the future can be seamlessly integrated.

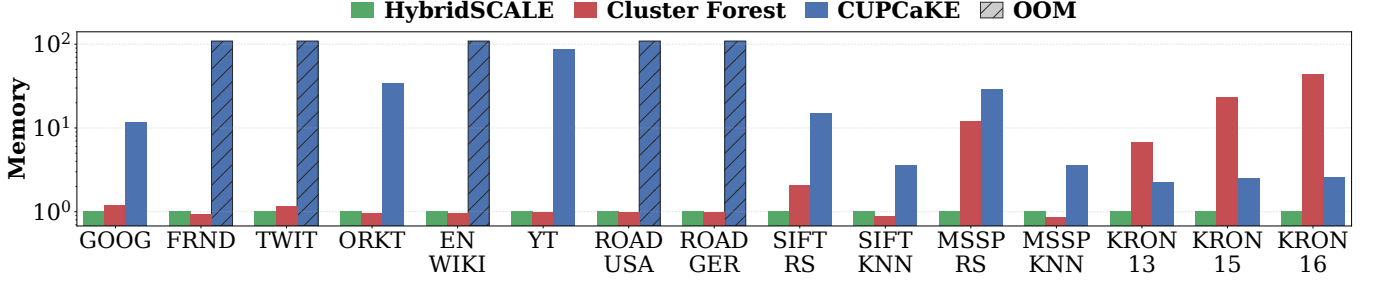


Figure 3: Peak memory usage of HYBRIDSCALE, Cluster Forest, and CUPCaKE on various input graph streams (normalized to HYBRIDSCALE). A bar with diagonal hashes indicates that the machine ran out of memory (OOM) while processing that input.

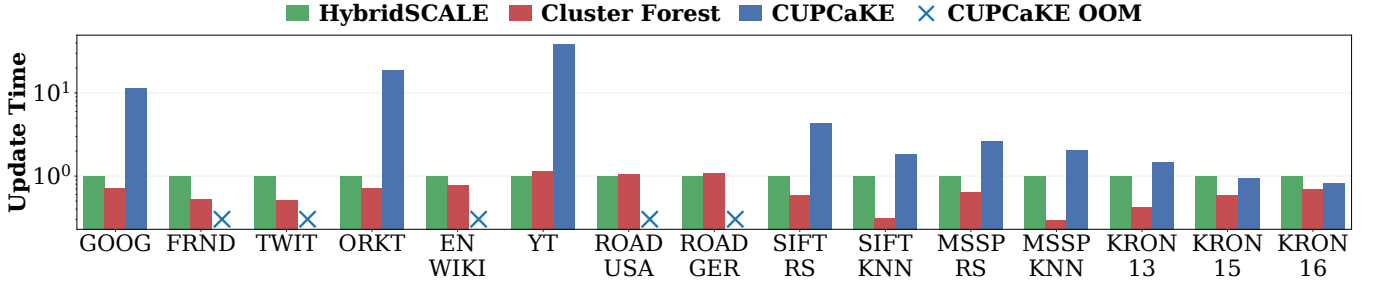


Figure 4: The total time taken by HYBRIDSCALE, Cluster Forest, and CUPCaKE to process all updates for various input graph streams (normalized to HYBRIDSCALE). The blue  $\times$  indicates that CUPCaKE ran out of memory (OOM) during initialization.

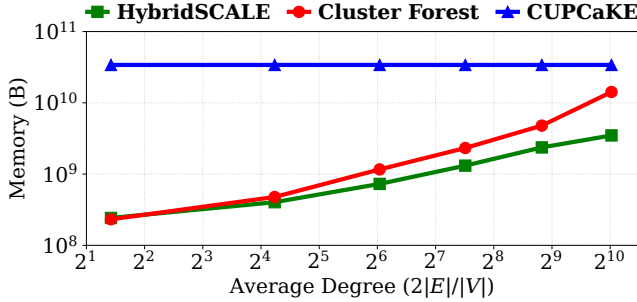


Figure 5: Memory usage HYBRIDSCALE, Cluster Forest, and CUPCaKE across various range search graphs constructed from a 1M-point subset of SIFT1B. The search radii are square Euclidean distances 10K, 20K, 30K, 40K, 50K, and 60K.

## 6.1 Memory Usage Results

For each dataset, we measure the maximum space used by each system at any point during stream ingestion. The results of this experiment are summarized in Figure 3. For each of the (sparse) social, web, and road graphs, the space used by HYBRIDSCALE is comparable to that of the Cluster Forest, and CUPCaKE uses one to two orders of magnitude more space. HYBRIDSCALE is 1.18 $\times$  more compact than Cluster Forest on the Google+ graph and 1.16 $\times$  more compact on the Twitter graph, because these graphs have high max  $k$ -core, indicating relatively large, dense cores. The other social, web, and

road graphs have smaller dense cores, leaving fewer opportunities to reduce the space cost by sketching. Consequently HYBRIDSCALE uses slightly more space than Cluster Forest due to the minor space overhead of our hybrid interface.

On the (dense) kron graphs, HYBRIDSCALE uses 1–2 orders of magnitude less space than Cluster Forest, because these graphs have very large average degree, making a lossless representation very large. Note that HYBRIDSCALE is also several times smaller than CUPCaKE, primarily due to the space improvements of BALLOONS-KETCH, and the usage of link-cut trees instead of Euler-tour trees.

On the SIFT1B and MSSPACEV1B datasets, we can further see the importance of core size and density to HYBRIDSCALE’s space performance. On these graphs with  $V = 10^6$ ,  $\delta$ , the HYBRIDSCALE density threshold is  $25 \cdot \lceil \log_2(10^6) \rceil = 500$ . In the  $knn$  graphs with  $k = 500$  each vertex has degree at least 500, therefore every vertex is included in the dense subsystem. However due to the properties of  $knn$  graphs, the max  $k$ -core is only 499. Thus these  $knn$  graphs represent a sort of worst-case input for the hybrid framework where every vertex is just dense enough to be sketched, but the dense region does not have a high enough average degree to achieve meaningful space savings through sketching. In contrast, the  $rs$  graphs have no limit on the degree of any vertex and tend to have larger, denser cores than their  $knn$  counterparts with similar average degree.

This core size discrepancy directly affects the space performance of HYBRIDSCALE. For the  $rs$  graphs, HYBRIDSCALE is significantly more space-efficient than Cluster Forest. Specifically, for SIFT1B HYBRIDSCALE uses a third of the space of Cluster Forest, and for MSSPACEV1B it uses an order of magnitude less space. For the  $knn$

graphs we observe that HYBRIDSCALE is moderately larger than Cluster Forest. Since it cannot save much space by sketching the nonexistent core, its overheads for the sketching and hybrid driver data structures make it a bit larger than Cluster Forest.

**Density Sweep.** Figure 5 further demonstrates the impact of core size, measuring the space usage of each system across rs graphs over the SIFT1B dataset with increasing squared euclidean distance radii. At low radius values, the dense core is too small to sketch efficiently and therefore HYBRIDSCALE and the Cluster Forest are essentially the same size. As the radius (and the size of the dense core) increases, HYBRIDSCALE becomes more compact than Cluster Forest. Note also that regardless of core size, HYBRIDSCALE is significantly smaller than CUPCaKE due to the reduced space complexity of BALLOONSKETCH. Due to this fact, HYBRIDSCALE can effectively leverage sketching to yield space savings at much lower average degrees than CUPCaKE (for CUPCaKE it must be in the low thousands).

## 6.2 Update Throughput Results

For each dataset, we measured the total time of each system to process all updates in the stream. The results are summarized in Figure 4. Our goal is to show that our hybrid framework incurs no significant degradation over the running time of its two subsystems. For example, on sparser graphs where few edges are sketched (e.g. the road graphs) the performance of HYBRIDSCALE is essentially the same as that of cluster forest. On the other hand, on graphs where most of the edges are sketched, (e.g. the knn and kron graphs), the update performance of HYBRIDSCALE is closer to that of CUPCaKE. For sparse graphs with reasonably dense cores (social and rs graphs), both subsystems of HYBRIDSCALE process a significant fraction of the edges, thus the running time is between Cluster Forest and CUPCaKE.

## 7 Conclusion

This work advances the story of practical graph sketching in several important ways. Sketch-based dynamic graph algorithms have a very different space usage profile from lossless ones: their space scales with the number of vertices rather than the number of edges, which is extremely attractive on dense graphs but imposes a significant per-vertex overhead that makes them very costly on sparse real-world graphs. This mismatch between where sketching shines in theory and where graphs actually sit in practice has long stood in the way of the broader adoption of graph sketching. Our central message of this paper is that sketching does not need to win on every graph in order to be useful. In particular, by applying sketching selectively to the dense cores of graphs and storing the rest losslessly, hybrid sketching can leverage sketching exactly where it helps most. Hybrid sketching is thus a simple “self-tuning” data structure that works well on graphs with densities ranging from very sparse graphs all the way toward very dense regimes where sketching offers massive advantages.

Hybrid sketching is a general approach, not a single algorithm. In particular, our system HYBRIDSCALE is agnostic to the specific lossless and sketch-based subroutines it wraps, so future work can freely substitute new components as the underlying algorithms improve. For instance, we could replace the lossless subsystem with a heuristic structure such as the ID-Tree [60], which trades theoretical guarantees for fast empirical performance on low-diameter graphs, or plug in future improved sketching primitives. Hybrid sketching should

also be naturally applicable to other graph streaming problems such as cut and spectral sparsifiers, spanners, triangle counting, and other sketch-based dynamic graph data structures. The broader takeaway of our work is that when used carefully, graph sketching already provides meaningful space savings for real-world graph datasets.

## Acknowledgments

We gratefully acknowledge support from NSF grants CCF-2247577, CCF-2403235, CNS-2317194, and NRT-HDR 2125295.

## References

- [1] Umut A. Acar, Daniel Anderson, Guy E. Blelloch, and Laxman Dhulipala. 2019. Parallel Batch-Dynamic Graph Connectivity. In *Proceedings of the 31st ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 381–392. <https://doi.org/10.1145/3323165.3323196>
- [2] Umut A. Acar, Daniel Anderson, Guy E. Blelloch, Laxman Dhulipala, and Sam Westrick. 2020. Parallel Batch-Dynamic Trees via Change Propagation. In *28th Annual European Symposium on Algorithms (ESA)*. 2:1–2:23. <https://doi.org/10.4230/LIPIcs.ESA.2020.2>
- [3] Kook Jin Ahn, Sudipto Guha, and Andrew McGregor. 2012. Analyzing graph structure via linear measurements. In *Proceedings of the twenty-third annual ACM-SIAM symposium on Discrete Algorithms*. SIAM, 459–467.
- [4] Kook Jin Ahn, Sudipto Guha, and Andrew McGregor. 2012. Graph sketches: sparsification, spanners, and subgraphs. In *PODS*. ACM, 5–14.
- [5] Noga Alon, Yossi Matias, and Mario Szegedy. 1999. The Space Complexity of Approximating the Frequency Moments. *J. Comput. System Sci.* 58, 1 (1999), 137–147. <https://doi.org/10.1006/jcss.1997.1545>
- [6] J. Ignacio Alvarez-Hamelin, Luca Dall’Asta, Alain Barrat, and Alessandro Vespignani. 2005. Large Scale Networks Fingerprinting and Visualization Using the  $k$ -Core Decomposition. In *Advances in Neural Information Processing Systems (NeurIPS)*. 41–50.
- [7] James A. Ang, Brian W. Barrett, Kyle B. Wheeler, and Richard C. Murphy. 2010. Introducing the Graph 500. In *Cray User Group (CUG) Proceedings*. [https://cug.org/5-publications/proceedings\\_attendee\\_lists/CUG10CD/pages/1-program/final\\_program/CUG10\\_Proceedings/pages/authors/11-15Wednesday/14C-Murphy-paper.pdf](https://cug.org/5-publications/proceedings_attendee_lists/CUG10CD/pages/1-program/final_program/CUG10_Proceedings/pages/authors/11-15Wednesday/14C-Murphy-paper.pdf)
- [8] Vladimir Batagelj and Matjaž Zaveršnik. 2003. An  $O(m)$  Algorithm for Cores Decomposition of Networks. *arXiv preprint cs/0310049* (2003).
- [9] Djamal Belazzougui, Gregory Kucherov, and Stefan Walzer. 2024. Better Space-Time-Robustness Trade-Offs for Set Reconciliation. In *51st International Colloquium on Automata, Languages, and Programming (ICALP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 297)*, Karl Bringmann, Martin Grohe, Gabriele Puppis, and Ola Svensson (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 20:1–20:19. <https://doi.org/10.4230/LIPIcs.ICALP.2024.20>
- [10] Paolo Boldi and Sebastiano Vigna. 2004. The WebGraph Framework I: Compression Techniques. In *Proceedings of the 13th International Conference on World Wide Web (WWW)*. ACM, 595–602. <https://doi.org/10.1145/988672.988752>
- [11] Stephen P. Borgatti and Martin G. Everett. 2000. Models of Core/Periphery Structures. *Social Networks* 21, 4 (2000), 375–395. [https://doi.org/10.1016/S0378-8733\(99\)00019-2](https://doi.org/10.1016/S0378-8733(99)00019-2)
- [12] Daniel G. Brown. 2011. How I Wasted Too Long Finding a Concentration Inequality for Sums of Geometric Variables. (2011).
- [13] CJ Carey, Jonathan Halcrow, Rajesh Jayaram, Vahab Mirrokni, Warren Schudy, and Peilin Zhong. 2022. Stars: Tera-Scale Graph Building for Clustering and Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [14] Shai Carmi, Shlomo Havlin, Scott Kirkpatrick, Yuval Shavitt, and Eran Shir. 2007. A Model of Internet Topology Using  $k$ -Shell Decomposition. *Proceedings of the National Academy of Sciences (PNAS)* 104, 27 (2007), 11150–11154. <https://doi.org/10.1073/pnas.0701175104>
- [15] Moses Charikar, Kevin Chen, and Martin Farach-Colton. [n. d.]. Finding Frequent Items in Data Streams. ([n. d.]).
- [16] Julia Chuzhoy, Yu Gao, Jason Li, Danupon Nanongkai, Richard Peng, and Thatchaphol Saranurak. 2020. A Deterministic Algorithm for Balanced Cut with Applications to Dynamic Connectivity, Flows, and Beyond. In *61st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*. 1158–1167. <https://doi.org/10.1109/FOCS46700.2020.00111>
- [17] Yann Collet. 2016. xxHash-Extremely fast non-cryptographic hash algorithm. URL <https://github.com/Cyan4973/xxHash> (2016).
- [18] Graham Cormode and Donatella Firmani. 2014. A unifying framework for lo-sampling algorithms. *Distrib. Parallel Databases* 32, 3 (sep 2014), 315–335. <https://doi.org/10.1007/s10619-013-7131-9>

- [19] Graham Cormode and Shanmugavelayutham Muthukrishnan. 2005. An Improved Data Stream Summary: The Count-Min Sketch and Its Applications. *Journal of Algorithms* 55, 1 (2005), 58–75. <https://doi.org/10.1016/j.jalgor.2003.12.001>
- [20] Timothy A. Davis and Yifan Hu. 2011. The university of Florida sparse matrix collection. *ACM Trans. Math. Softw.* 38, 1, Article 1 (dec 2011), 25 pages. <https://doi.org/10.1145/2049662.2049663>
- [21] Quinten De Man, Laxman Dhulipala, Adam Karczmarz, Jakub Łacki, Julian Shun, and Zhongqi Wang. 2024. Towards Scalable and Practical Batch-Dynamic Connectivity. *Proc. VLDB Endow.* 18, 3 (Nov. 2024), 889–901. <https://doi.org/10.14778/3712221.3712250>
- [22] Quinten De Man, Qamber Jafri, Daniel Delayo, Evan T. West, Michael A. Bender, and David Tench. 2025. Fast and Compact Sketch-Based Dynamic Connectivity. arXiv:2509.14433 [cs.DS] <https://arxiv.org/abs/2509.14433>
- [23] Quinten De Man, Atharva Sharma, Kishen N Gowda, and Laxman Dhulipala. 2026. UFO Trees: Practical and Provably-Efficient Parallel Batch-Dynamic Trees. In *Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming* (Sydney, NSW, Australia) (PPoPP '26). Association for Computing Machinery, New York, NY, USA, 109–122. <https://doi.org/10.1145/3774934.3786431>
- [24] Wei Dong, Moses Charikar, and Kai Li. 2011. Efficient k-nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th International Conference on World Wide Web (WWW)*.
- [25] Pavlos S. Efrimidis and Paul G. Spirakis. 2006. Weighted Random Sampling with a Reservoir. *Inform. Process. Lett.* 97, 5 (2006), 181–185. <https://doi.org/10.1016/j.ipl.2005.11.003>
- [26] David Eppstein, Zvi Galil, Giuseppe F. Italiano, and Amnon Nissenzweig. 1997. Sparsification—A Technique for Speeding Up Dynamic Graph Algorithms. *J. ACM* 44, 5 (1997), 669–696. <https://doi.org/10.1145/265910.265914>
- [27] Michalis Faloutsos, Petros Faloutsos, and Christos Faloutsos. 1999. On Power-Law Relationships of the Internet Topology. *ACM SIGCOMM Computer Communication Review* 29, 4 (1999), 251–262. <https://doi.org/10.1145/316194.316229>
- [28] Joan Feigenbaum, Sampath Kannan, Andrew McGregor, Siddharth Suri, and Jian Zhang. 2005. On Graph Problems in a Semi-Streaming Model. *Theor. Comput. Sci.* 348, 2 (Dec. 2005), 207–216. <https://doi.org/10.1016/j.tcs.2005.09.013>
- [29] Greg N. Frederickson. 1985. Data Structures for On-Line Updating of Minimum Spanning Trees, with Applications. *SIAM J. Comput.* 14, 4 (1985), 781–798. <https://doi.org/10.1137/0214055>
- [30] David Gibb, Bruce Kapron, Valerie King, and Nolan Thorn. 2015. Dynamic graph connectivity with improved worst case update time and sublinear space. arXiv:1509.06464 [cs.DS]
- [31] Michael T Goodrich and Michael Mitzenmacher. 2011. Invertible bloom lookup tables. In *2011 49th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*. IEEE, 792–799.
- [32] Monika R. Henzinger and Valerie King. 1999. Randomized Fully Dynamic Graph Algorithms with Polylogarithmic Time per Operation. *J. ACM* 46, 4 (1999), 502–516. <https://doi.org/10.1145/320211.320215>
- [33] Monika R. Henzinger and Mikkel Thorup. 1997. Sampling to Provide or to Bound: With Applications to Fully Dynamic Graph Algorithms. *Random Structures & Algorithms* 11, 4 (1997), 369–379. [https://doi.org/10.1002/\(SICI\)1098-2418\(199712\)11:4<369::AID-RSA5>3.0.CO;2-V](https://doi.org/10.1002/(SICI)1098-2418(199712)11:4<369::AID-RSA5>3.0.CO;2-V)
- [34] Jacob Holm, Kristian de Lichtenberg, and Mikkel Thorup. 2001. Poly-Logarithmic Deterministic Fully-Dynamic Algorithms for Connectivity, Minimum Spanning Tree, 2-Edge, and Biconnectivity. *J. ACM* 48, 4 (2001), 723–760. <https://doi.org/10.1145/502090.502095>
- [35] Jacob Holm, Eva Rotenberg, and Mikkel Thorup. 2018. Dynamic Bridge-Finding in  $\tilde{O}(\log^2 n)$  Amortized Time. In *Proceedings of the 29th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 35–52. <https://doi.org/10.1137/1.9781611975031.3>
- [36] Shang-En Huang, Dawei Huang, Tsvi Kopelowitz, Seth Pettie, and Mikkel Thorup. 2023. Fully Dynamic Connectivity in  $O(\log n (\log \log n)^2)$  Amortized Expected Time. *Theoretics* 2 (2023), 6:1–6:56. <https://doi.org/10.46298/theoretics.23.6>
- [37] Bruce M. Kapron, Valerie King, and Ben Mountjoy. 2013. Dynamic graph connectivity in polylogarithmic worst case time. In *Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 1131–1142. <https://doi.org/10.1145/777412.777415>
- [38] Richard M. Karp, Scott Shenker, and Christos H. Papadimitriou. 2003. A Simple Algorithm for Finding Frequent Elements in Streams and Its Application to Load Balancing. In *Proceedings of the 15th Annual ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 1–10. <https://doi.org/10.1145/777412.777415>
- [39] Maksim Kitsak, Lazaros K. Gallos, Shlomo Havlin, Fredrik Liljeros, Lev Muchnik, H. Eugene Stanley, and Hernán A. Makse. 2010. Identification of Influential Spreaders in Complex Networks. *Nature Physics* 6, 11 (2010), 888–893. <https://doi.org/10.1038/nphys1746>
- [40] Haewoon Kwak, Changhyun Lee, Hosung Park, and Sue Moon. 2010. What is Twitter, a Social Network or a News Media?. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*.
- [41] Julian J McAuley and Jure Leskovec. 2012. Learning to discover social circles in ego networks. In *NIPS*, Vol. 2012. Citeseer, 548–56.
- [42] Andrew McGregor. 2014. Graph stream algorithms: a survey. *ACM SIGMOD Record* 43, 1 (2014), 9–20.
- [43] Alan Mislove, Massimiliano Marcon, Krishna P. Gummadi, Peter Druschel, and Bobby Bhattacharjee. 2007. Measurement and Analysis of Online Social Networks. In *Proceedings of the 7th ACM SIGCOMM Conference on Internet Measurement (IMC)*. 29–42. <https://doi.org/10.1145/1298306.1298311>
- [44] Jayadev Misra and David Gries. 1982. Finding Repeated Elements. *Science of Computer Programming* 2, 2 (1982), 143–152. [https://doi.org/10.1016/0167-6423\(82\)90012-0](https://doi.org/10.1016/0167-6423(82)90012-0)
- [45] S. Muthukrishnan. 2005. Data Streams: Algorithms and Applications. *Foundations and Trends® in Theoretical Computer Science* 1, 2 (2005), 117–236. <https://doi.org/10.1561/0400000002>
- [46] Danupon Nanongkai, Thatchaphol Saranurak, and Christian Wulff-Nilsen. 2017. Dynamic Minimum Spanning Forest with Subpolynomial Worst-Case Update Time. In *IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*. 950–961. <https://doi.org/10.1109/FOCS.2017.92>
- [47] M. E. J. Newman. 2003. The Structure and Function of Complex Networks. *SIAM Rev.* 45, 2 (2003), 167–256. <https://doi.org/10.1137/S003614450342480>
- [48] OpenStreetMap contributors. 2017. Planet dump retrieved from <https://planet.osm.org>. <https://www.openstreetmap.org>.
- [49] Ryan A. Rossi and Nesreen K. Ahmed. 2015. The Network Data Repository with Interactive Graph Analytics and Visualization. In *AAAI*. <https://networkrepository.com>
- [50] Stephen B. Seidman. 1983. Network Structure and Minimum Degree. *Social Networks* 5, 3 (1983), 269–287. [https://doi.org/10.1016/0378-8733\(83\)90028-X](https://doi.org/10.1016/0378-8733(83)90028-X)
- [51] Moshe Shaked and J. George Shanthikumar (Eds.). 2007. *Stochastic Orders*. Springer, New York, NY. <https://doi.org/10.1007/978-0-387-34675-5>
- [52] Harsha Vardhan Simhadri, George Williams, Madlib-Abhimanyu, et al. 2021. Results of the NeurIPS’21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. In *NeurIPS 2021 Competitions and Demonstrations Track*. PMLR.
- [53] Daniel D Sleator and Robert Endre Tarjan. 1983. A data structure for dynamic trees. *J. Comput. System Sci.* 26, 3 (1983).
- [54] Daniel D. Sleator and Robert E. Tarjan. 1985. Self-Adjusting Binary Search Trees. *J. ACM* 32, 3 (1985), 652–686. <https://doi.org/10.1145/3828.3835>
- [55] David Tench, Evan West, Victor Zhang, Michael A. Bender, Abiyaz Chowdhury, J. Ahmed Dellas, Martin Farach-Colton, Tyler Seip, and Kenny Zhang. 2022. GraphZeppelin: Storage-Friendly Sketching for Connected Components on Dynamic Graph Streams. In *Proceedings of the 2022 International Conference on Management of Data (Philadelphia, PA, USA) (SIGMOD ’22)*. Association for Computing Machinery, New York, NY, USA, 325–339. <https://doi.org/10.1145/3514221.3526146>
- [56] David Tench, Evan T. West, Kenny Zhang, Michael Bender, Daniel DeLayo, Martin Farach-Colton, Gilvir Gill, Tyler Seip, and Victor Zhang. 2024. Exploring the Landscape of Distributed Graph Sketching. arXiv:2410.07518 [cs.DC] <https://arxiv.org/abs/2410.07518>
- [57] Mikkel Thorup. 2000. Near-Optimal Fully-Dynamic Graph Connectivity. In *Proceedings of the 32nd Annual ACM Symposium on Theory of Computing (STOC)*. 343–350. <https://doi.org/10.1145/335305.335345>
- [58] David P. Woodruff. 2014. Sketching as a Tool for Numerical Linear Algebra. *Foundations and Trends® in Theoretical Computer Science* 10, 1-2 (2014), 1–157. <https://doi.org/10.1561/0400000060> arXiv:1411.4357 [cs]
- [59] Christian Wulff-Nilsen. 2013. Faster deterministic fully-dynamic graph connectivity. In *Proceedings of the twenty-fourth Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM, 1757–1769.
- [60] Lantian Xu, Dong Wen, Lu Qin, Ronghua Li, Ying Zhang, and Xuemin Lin. 2024. Constant-time Connectivity Querying in Dynamic Graphs. *Proc. ACM Manag. Data* 2, 6, Article 230 (Dec. 2024), 23 pages. <https://doi.org/10.1145/3698805>
- [61] Jaewon Yang and Jure Leskovec. 2012. Defining and Evaluating Network Communities based on Ground-truth. In *IEEE International Conference on Data Mining (ICDM)*.
- [62] Shangdi Yu, Yan Gu, and Julian Shun. 2023. ParlayANN: Scalable and Deterministic Algorithms for Approximate Nearest Neighbor Search. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*.



## A BALLOONSKETCH Analysis

### A.1 Probability Preliminaries

#### A.1.1 Stochastic Dominance.

DEFINITION A.1 (STOCHASTIC DOMINANCE). *Let  $X$  and  $Y$  be random variables over  $\mathbb{R}$ . We use  $Y \succeq X$  to denote that  $Y$  **stochastically dominates**  $X$ , meaning:*

$$\Pr[Y \geq t] \geq \Pr[X \geq t],$$

for all  $t$  in  $\mathbb{R}$ .

This definition is equivalent to the statement that  $F_X(t) \geq F_Y(t)$  for all  $t$  in  $\mathbb{R}$  (note the flipped direction), where  $F_X(t)$  and  $F_Y(t)$  are the cumulative distribution functions of  $X$  and  $Y$ , respectively. Stochastically ordered variables have a number of useful closure properties, namely that they are closed under summation of independent variables:

LEMMA A.1. *Let  $X_1, X_2, \dots, X_n$  be a set of independent random variables, and let  $Y_1, Y_2, \dots, Y_n$  be another set of independent random variables such that  $X_i \preceq Y_i$  for all  $i \in \langle n \rangle$ . Then:*

$$\sum_{i=1}^n X_i \preceq \sum_{i=1}^n Y_i.$$

(Adapted from Theorem 1.A.3. in Shaked & Shanthikumar [51])

A.1.2 *Concentration Bounds.* We will make use of the following tail bound for geometric random variables:

LEMMA A.2. *Let  $\{Y_1, \dots, Y_n\}$  be the sum of  $n$  independent geometric random variables with success probability  $p$ , and  $k$  be some fixed constant. Define  $Y := \sum_{i=1}^n Y_i$ . Then  $Y$  is a negative binomially distributed random variable such that:*

$$\Pr[Y \geq k\mathbb{E}[Y]] \leq e^{-\frac{kn(1-1/k)^2}{2}}$$

A simple proof of this lemma can be found in [12].

### A.2 Bounding the Residual Depth of BALLOONSKETCH

Regardless of the level of independence of the hash function  $h$ , the residual depth of a geometric sampling sketch column  $C_h$  is itself bounded by a geometric random variable. This follows from union bounding the probability that a bucket  $b_i$ , or any entry below it, is non-empty, and then applying Markov's inequality to bound the probability that any bucket of depth  $i$  or greater is non-empty.

The formal statement and proof are given below in Lemma A.3.

LEMMA A.3. *Let  $M$  be the random variable corresponding to the residual depth of a sketch column  $C_h(x)$  with input  $x \in \mathbb{F}^n$ . Then  $M$  is stochastically dominated by a geometric random variable  $Y$  with  $p = 1/2$  and support  $\{1, 2, \dots\}$ .*

PROOF. Let  $Z := \text{supp}(x)$ , and  $m := \|x\|_0 = |Z|$ . We note that for all  $z \in Z$ , it holds, by the definition of a geometric sketch column, that

$$\Pr[z \in (Z \cap B_i)] = \Pr[z \in B_i] \leq \frac{1}{2^i}$$

Thus, the expected number of non-zeros in  $b_i$  is:

$$\begin{aligned} \mathbb{E}\left[\sum_{z \in Z} \mathbf{1}\{z \in B_i\}\right] &= \sum_{z \in Z} \mathbb{E}[\mathbf{1}\{z \in B_i\}] \quad (\text{linearity of expectation}) \\ &= \sum_{z \in Z} \Pr[z \in B_i] \\ &\leq \sum_{z \in Z} \frac{1}{2^i} = \frac{m}{2^i} \end{aligned}$$

We bound the probability that  $b_i$  is non-empty, by Markov's inequality:

$$\begin{aligned} \Pr[|B_i| > 0] &= \Pr\left[\left(\sum_{z \in Z} \mathbf{1}\{z \in B_i\}\right) \geq 1\right] \\ &\leq \mathbb{E}\left[\sum_{z \in Z} \mathbf{1}\{z \in B_i\}\right] \\ &\leq \frac{m}{2^i} \end{aligned}$$

Let  $w := \lceil \log_2 m \rceil$ . Then for all  $i \geq 0$ :

$$\begin{aligned} \Pr[|B_{w+i}| > 0] &\leq \frac{m}{2^{w+i}} \\ &\leq \frac{2^{\log_2 m}}{2^{\lceil \log_2 m \rceil}} \cdot \frac{1}{2^i} \\ &\leq \frac{1}{2^i} \end{aligned}$$

Let  $(G_1, G_2, \dots)$  be the sequence of random variables such that  $G_k$  is equal to 1 if there exists a non-empty bucket with depth greater than or equal to  $w+k$ , and 0 otherwise. Each  $G_k$  is a Bernoulli random variable with  $p = 2/2^{w+k}$ , and is 1 if and only if a non-empty bucket exists at or below  $b_{w+k}$ :

$$\begin{aligned} \Pr[G_k = 1] &:= \Pr\left[\bigvee_{j=k}^{\infty} (|B_{w+j+1}| > 0)\right] \leq \frac{1}{2^k} \sum_{j=0}^{\infty} \frac{1}{2^j} \\ &\leq \frac{2}{2^k} \\ \Pr[G_0 = 1] &\leq 1 \end{aligned}$$

Let  $X$  be a random variable equal to the smallest non-negative integer  $j$  such that  $b_{w+j}$  and every bucket in  $C_h$  below  $|b_{w+j}|$  are empty.

$$X := \min\{j \in \mathbb{Z}^+ \mid G_j = 0\}$$

This definition implies that  $w+j$  is the depth of  $C_h$ , and thus such a  $j$  would be the residual depth of  $C_h$ , unless the residual depth is equal to 0, in which case  $j$  is one more than the residual depth.

Accordingly,  $X$  is at most the residual depth of  $C_h$ , plus one. We can thus say that, if  $M$  is the R.V. for the residual depth,  $M \leq X$  statewise (this is a stronger condition than  $M \preceq X$ , as it implies that  $\Pr[X \geq k \mid M \geq k] = 1$ ).

Since  $G_b = 1 \implies G_a = 1$  and  $G_a = 0 \implies G_b = 0$  for all  $a \leq b$ :

$$M \geq k \implies X \geq k \iff \left( \bigwedge_{j=0}^k (G_j = 1) \right) \iff G_k = 1$$

thus:

$$\begin{aligned} \Pr[M \geq k] &\leq \Pr[X \geq k] = \Pr[G_k = 1] \leq \frac{2}{2^k} \\ \Pr[M \geq 0] &= 1 \end{aligned}$$

Let  $Y$  be a geometric random variable with  $p = 1/2$  and support  $\mathbb{Z}^+$ . Then:

$$\begin{aligned} \Pr[Y \geq k] &= \sum_{j=k}^{\infty} \frac{1}{2^j} \\ &= \frac{1}{2^k} \sum_{j=0}^{\infty} \frac{1}{2^j} \\ &= \frac{2}{2^k} \\ &\geq \Pr[M \geq k] \end{aligned}$$

for  $k \in \mathbb{Z}^+$ . We deal with the case that of  $k = 0$  separately (noting that, given how  $Y$  is defined, 0 is not typically in its support):

$$\begin{aligned} \Pr[Y \geq 0] &\geq \Pr[Y \geq 1] = 1 \\ &\geq \Pr[M \geq 0] \end{aligned}$$

Therefore, it holds for all  $t \in \mathbb{N}$  that  $\Pr[Y \geq t] \geq \Pr[M \geq t]$  (thus,  $Y \succeq M$ ).  $\square$

**COROLLARY A.4 (EXPECTED RESIDUAL DEPTH).** *Given a geometric sampling sketch column  $C_h$  with input  $a \in \mathbb{R}^n$ , the expected residual depth is  $\Theta(1)$ .*

**PROOF.** Let  $Y$  be a geometric random variable with  $p = 1/2$  and support  $\mathbb{Z}^+$ . Since  $Y \succeq M$  by Lemma A.3, it follows that  $\mathbb{E}[M] \leq \mathbb{E}[Y] \leq 2$ .  $\square$

### A.3 Aggregating Residual Depths Across Multiple Sketch Columns

Often, sketching algorithms use multiple sketch columns with independent randomness. The connectivity algorithms of AGM [3] (and its derivatives such as GRAPHZEPPELIN [55]), for example, require  $\Theta(\log n)$  sketches with constant success probability.

This independent randomness, combined with the properties of sums of independent geometrically distributed random variables, allows one to bound the sums of the residual depths for geometric sampling sketch columns both in expectation **and** with high probability, which in turn is also an upper bound on the total number of deep buckets.

Notably, as long as the hash functions  $h$  drawn for each  $C_h$  have independent randomness, the value of the sketched vector  $a$  does

not matter. It could be identical for all columns, as is the case during streaming ingestion for the AGM sketch and its variants.

**LEMMA A.5.** *Given a set of sketch columns  $\{C_{h_1}, \dots, C_{h_L}\}$ , the expected sum of residual depths is  $\Theta(L)$ .*

**PROOF.** Let  $M_i$  be the random variable for the residual depth of  $C_{h_i}$ . By linearity of expectation,

$$\begin{aligned} \mathbb{E}\left[\sum_{i=1}^L M_i\right] &= \sum_{i=1}^L \mathbb{E}[M_i] \\ &\leq \sum_{i=1}^L 2 \leq 2L \end{aligned} \quad (\text{Lemma A.3})$$

$\square$

**THEOREM A.6.** *Given a set of sketch columns  $\{C_{h_1}, \dots, C_{h_L}\}$  with independent randomness, the sums of residual depths across all  $C_{h_i}$  is at most  $\Theta(L + \log(n))$  with probability at least  $1 - 1/n^c$ .*

**PROOF.** Let  $M_i$  be the random variable for the residual depth of  $C_{h_i}$ . Let  $M$  be  $\sum_{i=1}^L M_i$ , the sum of residual depths across all columns. Let  $\{Y_1, \dots, Y_L\}$  be a set of independent geometric random variables with  $p = 1/2$  and support  $\mathbb{Z}^+$ . Let  $Y := \sum_{i=1}^L Y_i$ . By Lemma A.2,  $Y$  is a negative binomially distributed random variable such that:

$$\Pr[Y \geq k\mathbb{E}[Y]] \leq e^{-\frac{kL(1-1/k)^2}{2}}$$

Assume  $L \geq b \ln n$  for some constant  $b \in \mathbb{R}^+$ . Choosing  $k = 4bc$ , for  $c \geq 1$ , and noting (by linearity of expectation) that  $\mathbb{E}[Y] = 2L$ :

$$\begin{aligned} \Pr[Y \geq 8bcL] &\leq e^{-2bcL \cdot \frac{9}{16}} \\ &\leq \left(e^{bL}\right)^c \\ &\leq \frac{1}{n^c} \end{aligned}$$

Therefore, the probability that  $Y$  exceeds  $8bcL$  is at most  $1/n^c$ .

By Lemma A.3,  $M_i \preceq Y_i$  for all  $i$ . Since  $M = \sum M_i$  and  $Y = \sum Y_i$ , it follows from Lemma A.1 that  $M \preceq Y$ . Therefore,

$$\begin{aligned} \Pr[M \geq 8bcL] &\leq \Pr[Y \geq 8bcL] \\ &\leq 1/n^c \end{aligned} \quad (M \preceq Y)$$

Note that in the case that  $L < b \ln(n)$ , we can simply suppose we add an additional  $b \ln(n)$  geometric random variables to  $Y$ , and call this new variable  $\hat{Y}$ . From the tail bounds above it follows that  $\hat{Y}$  is  $\Theta(L + b \ln(n))$  with high probability, and since  $Y \leq \hat{Y}$  statewise, we conclude that  $Y$  is  $\Theta(L + b \ln(n))$  with at least that probability. The application to  $M$  via stochastic dominance follows exactly as above.  $\square$

**THEOREM 1.3 (BALLOONSKETCH).** *There exists an  $\ell_0$ -sampler, BALLOONSKETCH, such that given  $L$  independent BALLOONSKETCHES, each sketching the same vector  $x \in \mathbb{R}^n$  with  $m = \|x\|_0$ :*

- (1) *The total space usage of the sketches is  $O(L \log m + \log n)$  w.h.p.*
- (2) *The total cost of one update to each sketch is  $O(L + \log n)$  w.h.p.*

## A.4 BALLOONSKETCH

### Space and Time Complexity

PROOF. The total space for number of initialized non-deterministic buckets for a BALLOONSKETCH is equal to  $\lceil \log_2 m \rceil$  plus the residual depth. Theorem A.6 bounds the total residual depth as  $\Theta(L + \log(n))$  w.h.p..

The update cost for a single coordinate  $x$  is upper bounded by the sums of the depths derived from  $h_i(x)$  for the  $i$ -th BALLOONSKETCH; if the depth of the coordinate happens to be greater or equal to that of depth of the sketch, we may have to reallocate the BALLOONSKETCH to shrink. Let  $Y_i$  be the depth of a single update  $x$  in column  $i$ , this is stochastically dominated by some geometric random variable  $G_i$  by construction, and thus  $\sum_{i=1}^L Y_i = O(L + \log n)$  w.h.p. by an identical argument to the one found in Theorem A.6. Therefore, the update cost is  $O(L + \log n)$  w.h.p.

The correctness and sampling probability follow from the proof for CAMEOSKETCH found in [56].  $\square$

## B Hybrid Streaming Connectivity Proofs

LEMMA B.1. *The total space usage of the hybrid streaming algorithm is  $O(V + E)$  and  $O(V \log V \log(2 + E/V))$  w.h.p.*

PROOF. First we prove the  $O(V + E)$  w.h.p. space bound. We start by arguing that for each vertex  $v$  with degree  $d_v$ , it uses  $O(1 + d_v)$  space w.h.p. If  $v$  is in explicit form it requires  $\Theta(1 + d_v)$  space. If  $v$ 's adjacency list is stored as a BALLOONSKETCH matrix, then  $d_v > \delta/2$ , meaning  $d_v = \Omega(\log V \log \log V)$ . Per Theorem 1.3, and using  $L = \Theta(\log V)$ , the space usage is  $O(L \log d_v + \log V) = O(\log V \log d_v)$  w.h.p. Since  $d_v$  is at most  $V$ , the space used by a BALLOONSKETCH matrix for  $v$  is also  $O(1 + d_v)$  w.h.p. Summing over all vertices, the total space usage is:

$$\sum_{v \in V} O(1 + d_v) = O(V) + O\left(\sum_{v \in V} d_v\right) = O(V + E) \text{ w.h.p.}$$

Next we prove the  $O(V \log V \log(2 + E/V))$  w.h.p. space bound. In this case, we will argue that each vertex  $v$  with degree  $d_v$  uses  $O(\log V \log(2 + d_v))$  space w.h.p. For a vertex in explicit form, we have  $d_v \leq \delta = \Theta(\log V \log \log V)$ . At the threshold ( $d_v = \delta$ ), the set's space usage (BBST) is  $\Theta(\log V \log \log V)$ , which matches the desired bound of  $O(\log V \log(2 + d_v)) = O(\log V \log(2 + \log V \log \log V)) = O(\log V \log \log V)$ . For all degrees below the threshold ( $d_v < \delta$ ), the bound  $O(\log V \log(2 + d_v))$  remains valid because the ratio,  $R(d) = d/\log(2 + d)$ , is monotonically increasing for  $d \geq 1$ .

For a vertex in sketch form, we have  $d_v = \Omega(\log V \log \log V)$ . Per Theorem 1.3, the space of the BALLOONSKETCH matrix with  $L = \Theta(\log V)$  is  $O(\log V \log(2 + d_v))$  w.h.p. (since the number of nonzero elements in the sketched vector is  $m = d_v$ ). By a union bound over all  $V$  vertices, the total space usage is:

$$\sum_{v \in V} \log V \log(2 + d_v) = O(V) + O(\log V) \sum_{v \in V} \log(2 + d_v) \text{ w.h.p.}$$

Since  $\log(2 + x)$  is concave, applying Jensen's inequality:

$$\frac{1}{V} \sum_{v \in V} \log(2 + d_v) \leq \log\left(2 + \frac{1}{V} \sum_{v \in V} d_v\right) = \log\left(2 + \frac{2E}{V}\right).$$

Substituting this back into the total space expression:

$$O(V) + V \cdot O(\log V) \cdot \log\left(2 + \frac{2E}{V}\right) = O\left(V \log V \log\left(2 + \frac{E}{V}\right)\right) \text{ w.h.p.}$$

Finally, we note that the IBLT for a vertex  $v$  in sketch form uses  $\Theta(\log V \log \log V)$  space which is always dominated by the sketch space since  $d_v = \Omega(\log V \log \log V)$ , so the space used by the sketch is  $\Omega(\log V \log \log(2 + \log V \log \log V)) = \Omega(\log V \log \log V)$  w.h.p.  $\square$

LEMMA B.2. *The hybrid streaming algorithm processes each edge insertion and deletion in  $O(\log V)$  amortized time with high probability.*

PROOF. First, we bound the base cost of an update that does not trigger a vertex transition. As established, explicit sets are implemented as BBSTs, so insertion, deletion, and search times are all loosely upper bounded by  $O(\log V)$ . If the edge connects two sketch-form vertices, inserting or deleting the edge requires updating the BALLOONSKETCH matrix and IBLT of both endpoints, which takes  $O(\log V)$  time w.h.p. Thus, the base cost of any single stream update is bounded by  $O(\log V)$  w.h.p.

Next, we bound the computational cost of vertex promotions and demotions. A vertex  $u$  is promoted from explicit form to sketch form when its degree reaches  $\delta$ . Vertex  $u$  initializes a new sketch and IBLT, and redistributes its  $\delta$  explicitly stored edges. For each edge,  $u$  either adds it to its new sketch and issues an update to a sketch-form neighbor (costing  $O(\log V)$  w.h.p.), or pushes it to an explicit-form neighbor's set (costing  $O(\log V)$ ). Summing over all  $\delta$  edges, the total cost of a promotion is  $O(\delta \log V)$  w.h.p.

Similarly, a vertex  $u$  demotes from sketch form to explicit form when its degree drops to  $\delta/2$ . Vertex  $u$  decodes its IBLT to recover at most  $\delta/2$  edges (taking  $O(\delta)$  time), builds a new explicit set (taking  $O(\delta \log V)$  time), and issues deletions to its sketch-form neighbors (taking  $O(\delta \log V)$  time w.h.p.). The total cost of a demotion operation is  $O(\delta \log V)$  w.h.p.

Once a vertex is promoted or demoted, it must experience an absolute net change of at least  $\delta/2$  edge insertions or deletions before it can transition again. We can therefore distribute the  $O(\delta \log V)$  transition cost evenly across these  $\Omega(\delta)$  required stream updates, yielding an amortized cost of  $O(\log V)$  per update.  $\square$

**Connected Components Queries.** Our connected component query algorithm uses ideas from that of Ahn et al. (described in Section 2.2). However, their algorithm does not work directly because we have some vertices store their adjacency list in explicit form. Recall that the  $\ell_0$  sketching math requires every internal edge to be stored symmetrically by both endpoints so they cancel out via XOR summation. If an edge were stored explicitly by an array on one side and sketched on the other, it would fail to cancel, resulting in the query failing. Our asymmetric update rule was designed specifically to prevent this: by keeping mixed edges out of the sketches entirely and only sketching edges when both endpoints are in sketch form, we preserve the required mathematical parity. This allows us to safely execute the query using a multi-phase algorithm.

First, we run a standard spanning forest algorithm (e.g. BFS) over all edges stored in explicit form across the entire graph. This effectively collapses the graph into a set of disjoint super-components, such that any edges between them are stored as sketch-to-sketch edges in our hybrid algorithm. For each resulting super-component  $C$ , we generate its aggregate BALLOONSKETCH matrix by computing

the bitwise XOR sum of the sketches of all sketch-form vertices  $v \in C$ . Vertices in explicit-form contribute nothing to this sum. We treat the aggregated super-components as individual super-vertices and execute the standard Borůvka-style query algorithm of Ahn et al. on their aggregated BALLOONSKETCH structures to find the remaining cross-component edges and finalize the spanning forest. We prove in Appendix B that queries are correct w.h.p. and take  $O(V \log^2 V)$  time.

LEMMA B.3. *The hybrid streaming connected components query algorithm runs in  $O(V \log^2 V)$  time and is correct w.h.p.*

PROOF. We bound the time complexity by analyzing the multi-phase execution. The first phase runs a standard spanning forest algorithm (e.g., BFS) over the explicitly stored edges, which takes  $O(V + E_{\text{exp}})$  time. Because vertices only store explicit edges up to the density threshold  $\delta = \Theta(\log V \log \log V)$ , the total number of explicit edges is  $E_{\text{exp}} \leq V\delta$ . This BFS phase therefore strictly takes  $O(V \log V \log \log V)$  time. In the second phase, aggregating the sketches and executing the standard AGM-style Borůvka query takes at most  $O(V \log^2 V)$  time. This second phase dominates the runtime, giving an overall time complexity of  $O(V \log^2 V)$ .

For correctness, the first phase is deterministically correct: the BFS evaluates explicitly stored edges and perfectly groups the graph into super-components. The algorithm then relies on the standard AGM framework to resolve the remaining sketch-to-sketch edges. Because the  $\ell_0$  samplers guarantee the discovery of cross-component edges with high probability, the overall algorithm correctly outputs the final spanning forest with high probability.  $\square$

## C BALLOONSKETCH

### Dynamic Connectivity Analysis

LEMMA 4.1. *There exists a dynamic connectivity algorithm with space complexity  $O(V \log V \log(2 + E/V))$  w.h.p. The update complexity is  $O(\log^4 V)$ . The query complexity is  $O(\log V / \log \log V)$ .*

PROOF. We prove the space bound. Our data structure consists of  $T = \Theta(\log(V))$  tiers, matching the number needed in CUPCaKE, each with a UFO tree  $\mathcal{U}^t$  maintaining aggregates for a subtree query (where the aggregate is a single BALLOONSKETCH). For each UFO tree  $\mathcal{U}^t$ , there are levels  $\mathcal{U}^{0,t}, \mathcal{U}^{1,t}, \dots, \mathcal{U}^{\ell,t}, \dots, \mathcal{U}^{O(\log(V)),t}$ . Level  $\ell$  of a UFO tree has at most  $5/6$  as many nodes as level  $\ell - 1$ . Each level  $\ell$  consists of  $N_{\ell,t} \leq (5/6)^\ell V$  nodes, each of which with an aggregate BALLOONSKETCH. Each node represents a subset of the vertices, and the nodes at each level  $\ell$  collectively form a partitioning of the vertices.

Let  $U_i^{\ell,t}$  be the  $i$ th node at level  $\ell$  of the tier  $t$  UFO tree for  $1 \leq i \leq N_{\ell,t}$  (assigning an arbitrary ordering to the nodes at each level). Let  $d_i^{\ell,t}$  be the number of edges in the cut corresponding to  $(\mathcal{V}, \mathcal{V} \setminus U_i^{\ell,t})$ . Intuitively  $d_i^{\ell,t}$  is the “degree” (number of edges coming out) of the  $i$ -th cluster at level  $\ell$  in the UFO tree for tier  $t$ . The depth of the BALLOONSKETCH for  $U_i^{\ell,t}$  is:

$$\lg(d_i^{\ell,t}) + Y_i^{\ell,t}$$

where  $Y_i^{\ell,t}$  is a random variable (determined by the randomness of the sketch seed) that is stochastically dominated by a geometric random variable. The sum of depths for all internal UFO tree nodes across all  $T$  tiers is bounded by:

$$\sum_{t=0}^{T-1} \sum_{\ell=0}^{O(\log(V))} \sum_{i=1}^{N_{\ell,t}} (\lg(d_i^{\ell,t}) + Y_i^{\ell,t})$$

First, we show that the contribution from the  $\lg(d_i^{\ell,t})$  terms is  $O(V \log V \log(E/V))$ . For a fixed tier  $t$  and fixed level  $\ell$ , we know that  $N_{\ell,t} \leq (5/6)^\ell V$ . Let  $E_\ell \leq E$  be the number of **external** edges among our level  $\ell$  partition, i.e., the edges that exist between two different nodes at level  $\ell$  but not the edges internal to a node. Applying Jensen’s inequality:

$$\begin{aligned} D(\ell, t) &:= \sum_{i=1}^{N_{\ell,t}} \lg(d_i^{\ell,t}) \leq N_{\ell,t} \lg\left(\frac{\sum_{i=1}^{N_{\ell,t}} d_i^{\ell,t}}{N_{\ell,t}}\right) \\ &\leq N_{\ell,t} \lg\left(\frac{2E_\ell}{N_{\ell,t}}\right) \\ &\leq \alpha \left(\frac{5}{6}\right)^\ell V \lg\left(\frac{2E}{\alpha \left(\frac{5}{6}\right)^\ell V}\right) \\ &\leq \alpha \left(\frac{5}{6}\right)^\ell \left(1 - \lg(\alpha) + \ell \lg\left(\frac{6}{5}\right)\right) V \lg\left(\frac{2E}{V}\right) \end{aligned}$$

Where  $\alpha \in (0, 1)$  is an unknown real number such that  $N_{\ell,t} = \alpha(5/6)^\ell V$ . Upper bounding the r.h.s. with respect to  $\alpha$  gives

$$\begin{aligned} D(\ell, t) &\leq \alpha \left(\frac{5}{6}\right)^\ell \left(1 - \lg(\alpha) + \ell \lg\left(\frac{6}{5}\right)\right) V \lg\left(\frac{2E}{V}\right) \\ &\leq \left(\frac{5}{6}\right)^\ell \cdot \left(\alpha(1 - \lg(\alpha)) + \alpha \ell \lg\left(\frac{6}{5}\right)\right) V \lg\left(\frac{2E}{V}\right) \\ &\leq \left(\frac{5}{6}\right)^\ell \cdot \left(1.07 + \ell \lg\left(\frac{6}{5}\right)\right) V \lg\left(\frac{2E}{V}\right) \end{aligned}$$

The upper bounding of  $\alpha(1 - \lg(\alpha))$  was found analytically (the maximum is achieved at  $\alpha = 2/e$ ).

Across all levels  $\ell$  of a fixed tier  $t$ , the sum of depths of stored sketch aggregates is:

$$\begin{aligned} \sum_{\ell=0}^{O(\log V)} D(\ell, t) &\leq \sum_{\ell=0}^{O(\log V)} \left(\frac{5}{6}\right)^\ell \cdot \left(1.07 + \ell \lg\left(\frac{6}{5}\right)\right) V \lg\left(\frac{2E}{V}\right) \\ &\leq V \lg\left(\frac{2E}{V}\right) \cdot \sum_{\ell=0}^{O(\log V)} \left(\frac{5}{6}\right)^\ell \cdot \left(1.07 + \ell \lg\left(\frac{6}{5}\right)\right) \\ &\leq V \lg\left(\frac{2E}{V}\right) \cdot \left(1.07 \sum_{\ell=0}^{\infty} \left(\frac{5}{6}\right)^\ell + \lg\left(\frac{6}{5}\right) \sum_{\ell=0}^{\infty} \ell \left(\frac{5}{6}\right)^\ell\right) \\ &\leq V \lg\left(\frac{2E}{V}\right) \cdot \left(1.07 \cdot 6 + \lg\left(\frac{6}{5}\right) \cdot 30\right) \\ &\leq 15V \lg\left(\frac{2E}{V}\right) \end{aligned}$$

Summing across all  $T$  tiers:

$$\begin{aligned}
\sum_{t=0}^{T-1} \sum_{\ell=0}^{O(\log(V))} \sum_{i=1}^{N_{\ell,t}} \lg(d_i^{\ell,t}) &= \sum_{t=0}^{T-1} \sum_{\ell=0}^{O(\log(V))} D(\ell, t) \\
&\leq \sum_{t=0}^{T-1} 15V \log\left(\frac{2E}{V}\right) \\
&= O\left(V \log V \log\left(\frac{2E}{V}\right)\right)
\end{aligned}$$

Meanwhile, the sums of residual depths across all sketches is:

$$\sum_{t=0}^{T-1} \sum_{\ell=0}^{O(\log(V))} \sum_{i=1}^{N_{\ell,t}} Y_i^{\ell,t}$$

Let  $L$  be the maximum level across all  $T$  tiers and note that  $L = O(\log(V))$ . Let

$$\hat{Y}_i^{\ell,t} = \begin{cases} Y_i^{\ell,t} & \text{if UFO node } U_i^{\ell,t} \text{ exists} \\ 0 & \text{otherwise} \end{cases}$$

and note:

$$\begin{aligned}
\sum_{t=0}^{T-1} \sum_{\ell=0}^L \sum_{i=1}^{N_{\ell,t}} Y_i^{\ell,t} &= \sum_{t=0}^{T-1} \sum_{\ell=0}^L \sum_{i=1}^{(5/6)^\ell} \hat{Y}_i^{\ell,t} \\
&= \sum_{t=0}^L \sum_{i=1}^{(5/6)^\ell} \left( \sum_{t=0}^{T-1} \hat{Y}_i^{\ell,t} \right)
\end{aligned}$$

Focusing on a fixed  $\ell$  and  $i$  at the sum over the tiers, define new geometric random variables  $G_0, \dots, G_{T-1}$ . By the definition of  $Y_i^{\ell,t}$  it is clear that  $G_i \succeq \hat{Y}_i^{\ell,t}$ . Furthermore, since the randomness of  $\hat{Y}_i^{\ell,t}$  is determined by the tier  $t$  and each R.V. in the set  $\{\hat{Y}_i^{\ell,t} \mid 0 \leq t \leq T-1\}$  has a unique  $t$ , the set of  $\hat{Y}_i^{\ell,t}$  are mutually independent. Lemma A.1 then implies:

$$\sum_{t=0}^{T-1} \hat{Y}_i^{\ell,t} \preceq \sum_{t=0}^{T-1} G_t$$

and therefore:

$$\sum_{t=0}^{T-1} \sum_{\ell=0}^L \sum_{i=1}^{N_{\ell,t}} Y_i^{\ell,t} \preceq \sum_{\ell=0}^L \sum_{i=1}^{(5/6)^\ell} \left( \sum_{t=0}^{T-1} G_t \right)$$

By Lemma A.2, w.h.p.,  $\sum_{t=0}^{T-1} G_t = O(T + \log(V)) = O(\log(V))$  for a fixed  $\ell$  and  $i$ . Union bounding over all  $O(V)$  choices of  $(\ell, i)$ :

$$\sum_{t=0}^{T-1} \sum_{\ell=0}^L \sum_{i=1}^{N_{\ell,t}} Y_i^{\ell,t} = O(V \log(V))$$

with high probability, which is dominated by the  $O(V \log V \log(E/V))$  minimum depth implied by the  $\lg(d_i^{\ell,t})$  terms. Therefore, the total space usage is  $O(V \log V \log(E/V))$  with high probability.

The update bound and the query bound follow from the fact that the costs of LINK and CUT operations in UFO tree is asymptotically

equivalent to other dynamic tree data structures, and we do not modify the overall update and query algorithms of Gibb's algorithm.

For the sake of clean arithmetic we assumed WLOG that each vertex (and each cluster) has a degree of at least 2; if a vertex has degree zero, it still uses  $\Theta(1)$  words of space per each  $\Theta(\log V)$  tier, and thus the space is always at least  $\Omega(V \log V)$ . Our final space usage thus has a plus two added to the second logarithm to account for this, yielding  $O(V \log V \log(2 + E/V))$ .  $\square$

## D Hybrid Dynamic Connectivity Proofs

LEMMA D.1. *The space complexity of our hybrid dynamic connectivity algorithm is  $O(V + E)$  w.h.p.*

PROOF. First we prove the space is  $O(V + E)$  w.h.p.

Let  $V_D$  and  $E_D$  be the number of vertices and edges in  $ALG_D$  respectively. Let  $MEM_D$  be the random variable for the space usage of  $ALG_D$ , and  $MEM_S$  be the space usage of  $ALG_S$ .

Due to Lemma 4.1, we know that  $MEM_D = O(V_D \log V \log(2 + E_D/V_D))$  with high probability in  $V$ . If  $E_D \geq V_D \log V \log(2 + E_D/V_D)$ , then this space is  $O(E)$  with high probability since  $E \geq E_D$ . Otherwise, we have:

$$E_D \leq V_D \log V \log(2 + E_D/V_D) \leq 2V_D \log^2 V.$$

Therefore,

$$\begin{aligned}
MEM_D &= O(V_D \log V \log(2 + E_D/V_D)) \\
&= O(V_D \log V \log(2 + 2 \log^2 V)) \\
&= O(V_D \log V \log \log V)
\end{aligned}$$

with high probability.

Every vertex in  $ALG_D$  has degree at least  $\delta/2 = \log V \log \log V/2$ , therefore the total number of edges is  $E \geq V_D \log V \log \log V/4$ . Thus, in this case as well, the space in  $ALG_D$  is  $O(E)$  with high probability.

$MEM_S$  is  $O(V + E)$  since the graph stored by  $ALG_S$  is a subgraph of  $\mathcal{G}$ , and the Cluster Forest algorithm uses linear space. The space usage in our degree storage is clearly  $O(V)$ . The space usage of the  $V_D$   $\delta$ -sparse recovery sketches is  $O(V_D \log V \log \log V)$ , which again is  $O(E)$  since  $E \geq V_D \log V \log \log V/4$ . Therefore, the total space complexity  $MEM_D + MEM_S$  along with auxiliary structures is  $O(V + E)$  with high probability.

Finally the space usage in IBLT's is  $O(V_D \log V \log \log V)$  which is dominated by  $MEM_D$ . The boolean and global degree for each vertex only take  $O(V)$  total space.  $\square$

LEMMA D.2. *The space complexity of our hybrid dynamic connectivity algorithm is  $O(V \log V \log(2 + E/V))$  w.h.p.*

PROOF. Let  $V_D$  and  $E_D$  be the number of vertices and edges in  $ALG_D$  respectively. Let  $MEM_D$  be the random variable for the space usage of  $ALG_D$ , and  $MEM_S$  be the space usage of  $ALG_S$ .

We know that  $MEM_D = O(V_D \log V \log(2 + E_D/V_D))$  with high probability in  $V$ . Note that  $f(x) = x \log(2 + y/x)$  is a monotonically increasing function for positive  $x$  and a fixed positive  $y$ . Thus, since  $V_D \leq V$  and  $E_D \leq E$ , we have  $MEM_D = O(V \log V \log(2 + E/V))$  with high probability in  $V$ .

Because of the linear space complexity of the Cluster Forest algorithm,  $MEM_S = O(V + E_S)$ , where  $E_S$  is the number of edges in  $ALG_S$ . The spanning forest of  $ALG_D$  contributes at most  $V - 1$  edges

to  $E_S$ . The remaining edges must be sparse edges (the set of sparse edges may overcount the remaining edges in  $ALG_S$  because some sparse edges may be in  $ALG_D$  since we only demote vertices with degree  $< \delta/2$ , but no dense edges other than the spanning forest can be in  $ALG_S$ ).

To upper bound the number of sparse edges, we can simply sum the degrees of every sparse vertex. This results in edges between two sparse vertices being counted twice, and edges between a dense and a sparse vertex being counted once. Since every sparse vertex has degree  $< \delta$ , the number of sparse edges is  $< V\delta$ , so  $E_S < V \log V \log V$ .

If  $E < V \log V \log(2 + E/V)$  then  $MEM_S = O(V \log V \log(2 + E/V))$  is trivially true. Else  $E \geq V \log V \log(2 + E/V)$  and,

$$\begin{aligned} V \log V \log \log V &\leq V \log V \log \left( 2 + \frac{V \log V}{V} \right) \\ &\leq V \log V \log \left( 2 + \frac{V \log V \log(2 + E/V)}{V} \right) \\ &\leq V \log V \log \left( 2 + \frac{E}{V} \right). \end{aligned}$$

Therefore,  $E_S = O(V \log V \log(2 + E/V))$ , and  $MEM_S = O(V + E_S) = O(V \log V \log(2 + E/V))$ . Combining both bounds, the total space complexity  $MEM_D + MEM_S$  is  $O(V \log V \log(2 + E/V))$  with high probability in  $V$ .  $\square$

LEMMA D.3. *The amortized update complexity of our hybrid dynamic connectivity algorithm is  $O(\log^4 V)$ .*

PROOF. An update operation (edge insertion or deletion) in our hybrid framework consists of up to three main components: (1) updating the corresponding base algorithm ( $ALG_D$  or  $ALG_S$ ) and auxiliary structures, (2) applying any resulting spanning forest changes from  $ALG_D$  to  $ALG_S$ , and (3) handling potential vertex promotions or demotions.

First, consider the cost of a standard edge update that does not trigger a promotion or demotion. If the edge is in the sparse region, we update  $ALG_S$ . The Cluster Forest algorithm handles updates in amortized  $O(\log^2 V)$  time [59]. If the edge is in the dense region, we update  $ALG_D$ . Our BALLOONSKETCH-based algorithm (Section 4.1) supports updates in worst-case  $O(\log^4 V)$  time. Additionally, updating the IBLT's for the endpoints takes  $O(1)$  time. In the dense case,  $ALG_D$

may return a list of changes to its spanning forest. Sketch-based dynamic connectivity algorithms produce at most  $O(\log V)$  spanning forest changes per update. We must apply each of these changes to  $ALG_S$ , which takes  $O(\log V) \cdot O(\log^2 V) = O(\log^3 V)$  amortized time. Thus, the base cost of any single edge update, excluding vertex promotions/demotions, is bounded by  $O(\log^4 V)$  amortized.

Next, we analyze the cost of vertex promotions and demotions. When a vertex  $v$  is promoted (i.e., its degree reaches above  $\delta$ ), we iterate over its  $\delta$  incident edges in  $ALG_S$ . For any edge connecting to another heavy vertex, we insert it into  $ALG_D$  and delete it from  $ALG_S$ . This requires at most  $\delta$  operations in  $ALG_D$  and  $ALG_S$ , along with tracking the subsequent spanning forest changes. Because each standard edge move costs bounded by  $O(\log^4 V)$  amortized, the total cost of promoting a vertex is  $O(\delta \log^4 V)$  amortized. Similarly, when a vertex  $v$  is demoted (its degree falls down to  $\delta/2$ ), we recover its at most  $\delta/2$  incident edges from  $IBLT_v$  in  $O(\delta)$  time, delete them from  $ALG_D$ , and insert them into  $ALG_S$ . This process also involves at most  $\delta/2$  edge moves, bounding the demotion cost by  $O(\delta \log^4 V)$  amortized.

To bound the amortized cost, we use a standard potential argument. A vertex is promoted only when its degree increases above  $\delta$ , and demoted only when its degree decreases to  $\delta/2$ . Therefore, between any two consecutive threshold crossings (a promotion followed by a demotion, or vice versa), the vertex must have undergone at least  $\delta/2$  individual edge insertions or deletions incident to it. We charge the  $O(\delta \log^4 V)$  cost of the promotion or demotion evenly across these  $\Omega(\delta)$  updates. This adds an amortized cost of  $O(\delta \log^4 V / \delta) = O(\log^4 V)$  to each update.

Combining the base update cost and the amortized promotion and demotion cost, the overall amortized update complexity of our hybrid framework is  $O(\log^4 V)$ .  $\square$

LEMMA D.4. *The worst-case query complexity of our hybrid dynamic connectivity algorithm is  $O(\log V / \log \log V)$ .*

PROOF. As described in our query algorithm (Section 4.2), any connectivity query  $\text{CONNECTED}(u, v)$  in our framework is directly answered by calling  $ALG_S.\text{CONNECTED}(u, v)$ . Because we use the Cluster Forest algorithm [59] as  $ALG_S$ , which supports worst-case  $O(\log V / \log \log V)$  query times, the query complexity of our hybrid framework is exactly  $O(\log V / \log \log V)$  in the worst case.  $\square$

## E Additional Experimental Results

Table 2 shows the un-normalized results of our memory usage experiments. Table 3 shows the un-normalized results of our update speed experiments.

Table 4 indicates the number of cutset tiers that were actually required for HYBRIDSCALE and CUPCaKE. The results show that the number of tiers is always lower than the number we allocated ( $\lceil \log_2 V \rceil$ ), indicating that the Monte Carlo random algorithms never induced an error in our experiments. Also HYBRIDSCALE can require significantly fewer tiers than CUPCaKE.

| Graph           | HYBRID<br>SCALE<br>(GB) | Cluster<br>Forest<br>(GB) | CUP<br>CaKE<br>(GB) |
|-----------------|-------------------------|---------------------------|---------------------|
| Google+         | 0.241                   | 0.284                     | 2.83                |
| Friendster      | 65.7                    | 60.7                      | —                   |
| Twitter         | 38.5                    | 44.5                      | —                   |
| Orkut           | 3.75                    | 3.54                      | 128                 |
| ENWiki          | 3.83                    | 3.62                      | —                   |
| Youtube         | 0.494                   | 0.479                     | 42.8                |
| RoadUSA         | 10.2                    | 9.93                      | —                   |
| RoadGER         | 5.33                    | 5.2                       | —                   |
| SIFT-RS-50K     | 2.33                    | 4.79                      | 34.9                |
| SIFT-KNN-500    | 10.1                    | 8.86                      | 36                  |
| MSSPACE-RS-10K  | 1.21                    | 14.4                      | 34.7                |
| MSSPACE-KNN-500 | 10                      | 8.54                      | 36                  |
| kron-13         | 0.0651                  | 0.439                     | 0.145               |
| kron-15         | 0.299                   | 6.98                      | 0.743               |
| kron-16         | 0.643                   | 27.9                      | 1.66                |

**Table 2: Peak memory usage (GB) in our experiments. Entries indicated as — ran out of memory (OOM).**

| Graph           | HYBRID<br>SCALE<br>(sec) | Cluster<br>Forest<br>(sec) | CUP<br>CaKE<br>(sec) |
|-----------------|--------------------------|----------------------------|----------------------|
| Google+         | 38.7                     | 27.5                       | 440                  |
| Friendster      | 42.2K                    | 22.2K                      | —                    |
| Twitter         | 22.9K                    | 11.8K                      | —                    |
| Orkut           | 902                      | 642                        | 16.7K                |
| ENWiki          | 638                      | 501                        | —                    |
| Youtube         | 18                       | 20.8                       | 697                  |
| RoadUSA         | 263                      | 274                        | —                    |
| RoadGER         | 163                      | 178                        | —                    |
| SIFT-RS-50K     | 1.75K                    | 1.03K                      | 7.52K                |
| SIFT-KNN-500    | 7.59K                    | 2.36K                      | 14K                  |
| MSSPACE-RS-10K  | 3.88K                    | 2.47K                      | 10.3K                |
| MSSPACE-KNN-500 | 6.61K                    | 1.93K                      | 13.5K                |
| kron-13         | 66.7                     | 28.2                       | 99.2                 |
| kron-15         | 1.64K                    | 961                        | 1.53K                |
| kron-16         | 8.41K                    | 5.87K                      | 6.98K                |

**Table 3: Total update time in seconds in our experiments. Entries indicated as — ran out of memory (OOM).**

| Graph           | $\lceil \log_2 V \rceil$ | HYBRID<br>SCALE | CUP<br>CaKE |
|-----------------|--------------------------|-----------------|-------------|
| Google+         | 17                       | 11              | 14          |
| Friendster      | 26                       | 14              | —           |
| Twitter         | 26                       | 15              | —           |
| Orkut           | 22                       | 11              | 18          |
| ENWiki          | 23                       | 12              | —           |
| Youtube         | 21                       | 9               | 15          |
| RoadUSA         | 25                       | 0               | —           |
| RoadGER         | 24                       | 0               | —           |
| SIFT-RS-50K     | 20                       | 13              | 17          |
| SIFT-KNN-500    | 20                       | 14              | 18          |
| MSSPACE-RS-10K  | 20                       | 13              | 16          |
| MSSPACE-KNN-500 | 20                       | 15              | 18          |
| kron-13         | 13                       | 11              | 12          |
| kron-15         | 15                       | 12              | 14          |
| kron-16         | 16                       | 14              | 14          |

**Table 4: Max number of tiers used. Entries indicated as — ran out of memory (OOM). A value of 0 indicates that HYBRIDSCALE did not sketch anything.**